BeeReady

Digitális produktivitási és tanulástámogató alkalmazás

Készítette:

Székely Balázs Székely Hunor

2025 – 2026

Tartalomjegyzék

TARTALOMJEGYZÉK **2**

1 BEVEZETÉS **8**

1.1 TÉMAVÁLASZTÁS 8

1.2 CÉLKITŰZÉS 9

1.3 KÜLÖN KÖSZÖNET 9

2 FEJLESZTŐI DOKUMENTÁCIÓ **10**

2.1 KIALAKÍTÁS ÉS TERVEZÉS 10

2.2 A BEEREADY MAPPA ADATSZERKEZETE 10

2.3 ADATBÁZIS LEÍRÁSA 11

2.3.1 Felhasználók és Adminok *11*

2.3.2 Tanulókártya (Flashcards) *12*

2.3.3 Kvíz (Quizzes) *13*

2.3.4 Eredmények, Megosztások és Kedvencek *15*

2.3.5 Szervezés és Időmenedzsment *16*

2.4 MEGVALÓSÍTÁSOK 18

2.4.1 Bejelentkezés és Regisztráció *18*

2.4.1.1 Avatár-választó modal 19

2.4.1.2 Regisztrációs form – beküldés és API hívás 19

2.4.1.3 Bejelentkezési form – JWT és az Emlékezz rám opció 19

2.4.2 Kettős Validáció *20*

2.4.3 Központi megvalósítások frontenden *20*

2.4.3.1 Központosított response kezelés 20

2.4.3.2 Értesítés megjelenítése az alertell(text, time) metódus 21

2.4.3.3 Kijelentkeztetési modal 21

2.4.3.4 Munkamenet kezelés és lejárat 22

2.4.4 Sortable.js könyvtár *22*

2.4.5 Főoldal *22*

2.4.5.1 Teendőlista (ToDo) 23

2.4.5.2 Időzítő 23

2.4.5.3 Naptár 24

2.4.5.4 Statisztika 24

2.4.6 Tanulókártyák *24*

2.4.6.1 Pakli lista nézet (#decks) 24

2.4.6.2 Kártya lista nézet (#cards) 25

2.4.6.3 Játék nézet (#flashcardGameContainer) 25

2.4.7 Órarend *25*

2.4.7.1 Nézet és DOM struktúra 25
2.4.7.2 Hétválasztó 26
2.4.7.3 Az aktuális nap megjelölése 26
2.4.8 Kvíz 26
2.4.8.1 A saját illetve a nem saját tulajdonú kvízek listázása 27
2.4.8.2 A kvízek létrehozása 27
2.4.8.2.1 Kérdéstípus-választó modál – showQuizTypeSelector() 28
2.4.8.2.2 A kvíz elmentése 29
2.4.8.2.3 A kitöltő feladattípus mentése 29
2.4.8.3 A kvízek szerkesztése 30
2.4.8.3.1 Kitöltő típus feladat betöltése 30
2.4.8.4 A kvízek kitöltése 31
2.4.8.4.1 Kvíz beküldése 31
2.4.8.4.2 A kitöltés kiértékelése 31
2.4.8.4.3 Közös lezárás – save_result() 33
2.4.8.5 A kitöltés eredményének megtekintése 33
2.4.9 Hive 34
2.4.9.1 Tartalom betöltése és megjelenítése 34
2.4.9.2 Szűrők és keresés 34
2.4.9.3 Lábléc-gombjai 35
2.4.10 Profil kezelés 35
2.4.11 Navigációs sáv 35
2.4.11.1 Modulok közötti navigáció (navbar_click) 36
2.4.11.2 Mobilnézet (hamburger menü) 36
2.4.12 Admin felület 36
2.5 BACKEND MEGVALÓSÍTÁS 37
2.5.1 Központosított hibakezelés 37
2.5.2 Regisztráció és belépés végpontok 37
2.5.3 Jelszóvalidálás – passwordTest() 37
2.5.4 Biztonság 37
3 TESZTELÉS ÉS TESZTDOKUMENTÁCIÓ 38
3.1 HIBANAPLÓ 38
3.1.1 ErrorHandler futási hiba 38
3.1.2 Jogosulatlan adathozzáférés 38
3.1.3 Kártya nélküli paklik kezelése és navigációs gomb állapot hiba 39
3.1.4 Accessing non-existent property 'isexistscheck' 40
3.1.5 ToDo törlés – confirm() nem jelenik meg 40
3.1.6 Hive - Keresőmező debounce – Felesleges hálózati kérések és globális változó kezelése 41
3.2 EGYSÉGTESZTEK 42

3.2.1 Tesztelési célkitűzések ... *42*
3.2.2 Alkalmazott technológiák ... *42*
3.2.3 Tesztelt komponensek leírása ... *42*
3.2.4 Mockolási stratégia ... *42*
3.2.5 Tesztek futtatása ... *42*
3.2.6 Tesztelési eredmények összefoglalása ... *43*
4 TOVÁBBFEJLESZTÉSI LEHETŐSÉGEK ... **44**
4.1 RÖVID TÁVÚ FEJLESZTÉSEK (KB. 1 HÓNAP) ... 44
4.1.1 Saját profilkép feltöltése ... *44*
4.1.2 Sötét mód (Dark mode) ... *44*
4.2 KÖZEPES TÁVÚ FEJLESZTÉSEK (1–3 HÓNAP) ... 44
4.2.1 Haladáskövetés és statisztikák ... *44*
4.2.2 Flashcard – "Ment / Nem ment" szűrő ... *44*
4.2.3 Hive (közösségi) modul bővítése ... *45*
4.2.4 Barát jelölés és profil megtekintés ... *45*
4.3 HOSSZÚ TÁVÚ FEJLESZTÉSEK (3+ HÓNAP) ... 45
4.3.1 Gamifikáció – XP, szintek, jelvények ... *45*
4.3.2 Tanár–diák szerepkörök ... *45*
4.3.3 Barátokkal chat ... *45*
4.3.4 Csoportos tanulás – Szobák ... *46*
5 FELHASZNÁLÓI DOKUMENTÁCIÓ ... **47**
5.1 LÉNYEGESEBB FUNKCIÓK ... 47
5.1.1 Főoldal ... *47*
5.1.1.1 Teendőlista ... 47
5.1.1.1.1 Feladat hozzáadása ... 47
5.1.1.1.2 Aktív feladatok kezelése ... 47
5.1.1.1.3 Kész feladatok kezelése\ ... 47
5.1.1.1.4 Statisztika widget ... 47
5.1.1.1.5 A tanulási nézet használata ... 48
5.1.1.2 Időzítő ... 48
5.1.1.2.1 Stopper mód ... 48
5.1.1.2.2 Pomodoro mód ... 48
5.1.1.2.3 Egyéni mód: ... 48
5.1.1.2.4 Gombok: ... 48
5.1.1.2.5 Tanulással töltött idő ... 48
5.1.1.3 Naptár ... 48
5.1.1.3.1 Navigáció ... 48
5.1.1.3.2 Esemény megtekintése és hozzáadása: ... 49

5.1.1.3.3 Esemény törlése: 49
5.1.1.3.4 Technikai korlátok: 49
5.1.2 Tanulókártyák 49
5.1.2.1 Paklik kezelése 49
5.1.2.1.1 Paklik létrehozása 49
5.1.2.1.2 Pakli sorrendjének megváltoztatása 50
5.1.2.1.3 Pakli szerkesztése 50
5.1.2.1.4 Pakli törlése 50
5.1.2.2 Kártyák kezelése 50
5.1.2.2.1 Pakli megnyitása 50
5.1.2.2.2 Új kártya hozzáadása 51
5.1.2.2.3 Kártya szerkesztése 51
5.1.2.2.4 Kártya törlése 51
5.1.2.2.5 Kártyák sorrendjének megváltoztatása 51
5.1.2.3 Tanulási mód 51
5.1.2.3.1 A tanulás megkezdése 51
5.1.2.3.2 A tanulási nézet használata 52
5.1.2.4 Paklik megosztása 52
5.1.2.4.1 Megosztási kód megismerése 52
5.1.2.4.2 Más pakli másolása megosztási kóddal 52
5.1.2.4.3 Megosztási kód cseréje 53
5.1.2.4.4 Publikus pakli beállítása 53
5.1.2.5 Technikai Korlátok 53
5.1.3 Órarend 53
5.1.3.1 Hetek kezelése ("A" és "B" hét) 54
5.1.3.2 Esemény (tanóra) hozzáadása 54
5.1.3.3 Meglévő esemény módosítása 54
5.1.3.4 Esemény törlése 54
5.1.3.5 Használat mobiltelefonon 55
5.1.3.6 Technikai korlátok 55
5.1.4 Kvíz 55
5.1.4.1 Kvízek kezelése 56
5.1.4.2 Kvíz létrehozása és szerkesztése 56
5.1.4.2.1 Kérdéstípusok 56
5.1.4.3 Kvíz kitöltése 57
5.1.4.4 Eredmények megtekintése 57
5.1.4.5 Törlés 58
5.1.4.6 Technikai korlátok 58
5.1.5 Hive 58
5.1.5.1 Tartalom böngészése 58
5.1.5.2 Szűrők és keresés 59
5.1.5.3 Mentés és törlés 59

5.1.5.4 Tartalom indítása 59
5.1.6 Profil kezelés 59
5.1.6.1 Módosítható adatok 59
5.1.6.2 Profilkép (Avatar) választása 59
5.1.6.3 Biztonság és ellenőrzés 60
5.1.7 Admin felület 60
5.1.7.1 A felület felépítése 60
5.1.7.2 Felhasználók keresése és listázása 60
5.1.7.3 Felhasználói adatok módosítása és törlése 60
5.1.7.4 Adminisztrátorok kezelése 60
5.1.7.4.1 Új adminisztrátor hozzáadása: 61
5.1.7.4.2 Adminok listája: 61
5.1.7.5 Biztonsági figyelmeztetések 61
5.2 HARDVER ÉS SZOFTVER SZÜKSÉGLETEK 61
5.2.1 Szerver oldal 61
5.2.2 Adatbázis 61
5.2.3 Kliens oldal 61
5.2.4 Hálózat 62
5.2.5 Szoftverkörnyezet (fejlesztéshez) 62
5.3 TELEPÍTÉS ÉS INDÍTÁS 62
5.3.1 Rendszerkövetelmények 62
5.3.2 Függőségek 62
5.3.2.1 Éles (production) függőségek 62
5.3.2.2 Fejlesztői (devDependencies) függőségek 63
5.3.3 Telepítési útmutató 64
5.3.4 Az alkalmazás indítása 65
5.3.4.1 Éles mód 65
5.3.4.2 Fejlesztői mód (nodemon – automatikus újraindítás): 65
5.4 RÉSZLETES BEMUTATÁS 65
6 CSAPATMUNKA MEGVALÓSÍTÁSA 66
6.1 MUNKAMEGOSZTÁS 66
6.2 VERZIÓKEZELÉS ÉS EGYÜTTMŰKÖDÉS 66
7 ÖSSZEFOGLALÁS 67
7.1 LEÍRÁS 67
7.2 FEJLESZTÉSI ESZKÖZÖK 68
7.3 MODULOK ÉS FUNKCIÓK 68
7.4 MINŐSÉGBIZTOSÍTÁS 68
7.5 TOVÁBBFEJLESZTÉSI IRÁNYOK 68

8 ÁBRAJEGYZÉK 69

9 IRODALOMJEGYZÉK (FORRÁSOK) 70

9.1 BACKEND KERETRENDSZER ÉS SZERVER 70

9.2 ADATBÁZIS ÉS KAPCSOLATKEZELÉS 70

9.3 HITELESÍTÉS, JWT ÉS JELSZÓKEZELÉS 70

9.4 RATE LIMITING ÉS HTTP NAPLÓZÁS 71

9.5 FRONTEND – CSS KERETRENDSZER 71

9.6 DRAG-AND-DROP RENDEZÉS – SORTABLE.JS 71

9.7 TESZTELÉS – JEST 71

9.8 FEJLESZTŐI SEGÉDESZKÖZÖK 72

9.9 WEBES SZABVÁNYOK ÉS REFERENCIÁK 72

9.10 EGYÉB FELHASZNÁLT ESZKÖZÖK ÉS INSPIRÁCIÓ 72

1 Bevezetés

1.1 Témaválasztás

A BeeReady egy modern, digitális felület kialakítására irányul, amely a mindennapi szervezést, feladatkezelést és tanulási folyamatokat kívánja hatékonyabbá tenni. A projekt alapötlete egy olyan rendszer létrehozása, amely ötvözi a produktivitási eszközök legfontosabb elemeit: az egyszerű kezelhetőséget, valamint az átlátható felületet. A fejlesztés célja egy letisztult, felhasználóbarát alkalmazás megalkotása, amely támogatja a felhasználókat a határidők, célok és teendők következetes és motiváló kezelésében.

A mai digitális környezetben egyre nagyobb szerepet kapnak azok a platformok, amelyek segítséget nyújtanak az időmenedzsmentben és a tanulási hatékonyság növelésében. A BeeReady ezekre az igényekre reflektál: biztosítja a felhasználók számára, hogy rendszerezetten, tudatosabban és fókuszáltabban végezhessék tevékenységeiket. Ezáltal nem csupán egy egyszerű alkalmazásról beszélhetünk, hanem a mindennapi munkavégzést támogató, komplex eszközről.

A digitális oktatás és a távmunka terjedésével egyre nagyobb hangsúly kerül a strukturáltságra és az önálló időgazdálkodásra. A projekt témája ezért nemcsak aktuális, hanem releváns is, hiszen egy olyan területet céloz meg, amely a jövőben is meghatározó marad. A BeeReady a technológiai fejlődés adta lehetőségeket szeretné kihasználni, hogy megkönnyítse a felhasználók életét és növelje hatékonyságukat.

Aktív tanulókként nap mint nap szembesülünk a tanulás nehézségeivel, a hatalmas mennyiségű tananyag rendszerezésével és a fókusz megtartásának problémájával. Úgy éreztük, hogy a hagyományos módszerek már nem nyújtanak elegendő támogatást a digitális zajban, ezért döntöttünk egy olyan eszköz fejlesztése mellett, amely közvetlenül ezekre a nehézségekre reflektál.

1.2 Célkitűzés

A BeeReady projekt fő célja egy olyan alkalmazás létrehozása, amely egyszerűen használható, ugyanakkor professzionális szintű funkciókat kínál. A célkitűzések között szerepel egy olyan felület megtervezése, amely támogatja:

- a feladatok és határidők gyors rögzítését
- a folyamatok és előrehaladás vizuális megjelenítését
- a felhasználói motiváció növelését

Emellett a projekt célja az is, hogy a BeeReady ne csupán technikailag működőképes legyen, hanem esztétikailag is modern, letisztult és könnyen átlátható. A jövőben bővíthető rendszerként kell funkcionálnia, amely képes alkalmazkodni a felhasználók változó igényeihez. A hosszú távú cél egy olyan platform kialakítása, amely mind tanulók, mind tanárok számára valódi segítséget jelent a mindennapi feladataik rendszerezésében és elvégzésében.

1.3 Külön köszönet

Szeretnénk köszönetet mondani mindazoknak, akik szakmai útmutatásukkal, támogatásukkal és építő jellegű kritikáikkal hozzájárultak projektünk sikeres megvalósításához.

Luczek Zsuzsa tanárnőnek a legmélyebb hálánkat fejezzük ki a projekt egésze alatt nyújtott kiemelkedő támogatásáért. Mentorálása, türelme és folyamatos bátorítása meghatározó volt számunkra. Szakmai meglátásai nélkül ez a munka nem érhette volna el jelenlegi formáját.

Ezúton köszönjük meg Csontos Péter tanár úrnak a munkánkhoz nyújtott szakmai iránymutatását. Köszönettel tartozunk a konzultációkért, frontend és backend oktatása miatt, valamint azokért az észrevételekért, amelyekkel segített a fejlesztést a megfelelő mederben tartani.

Köszönjük Bődi Ádám tanár úr a fejlesztés során nyújtott értékes segítségét, a közös gondolkodást, az adatbázissal lévő kérdéseink megválaszolását, és azokat a technikai észrevételeket, amelyekkel segített átlendülni a kritikus pontokon.

2 Fejlesztői dokumentáció

2.1 Kialakítás és tervezés

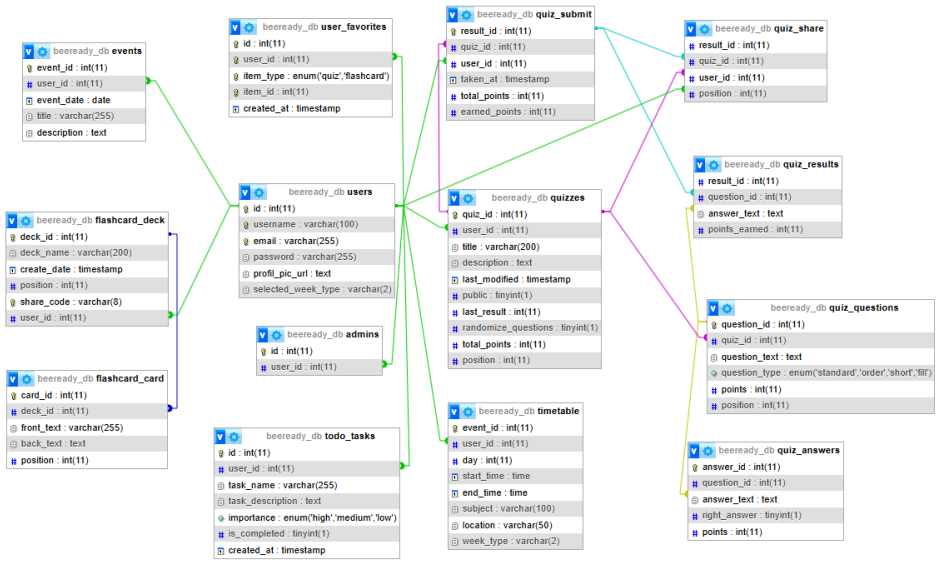
Az alkalmazás tervezése során kiemelkedő figyelmet fordítottunk a felhasználói élményre, a moduláris felépítésre és a könnyű bővíthetőségre. A rendszer egy kliens–szerver architektúrán alapul, ahol a frontend és a backend egyértelműen elválasztott rétegekként működnek.

2.2 A BeeReady mappa adatszerkezete

```
C:.
├── .gitignore        Git verziókezelési kivételek (pl. node_modules, .env)
├── README.md         Útmutató a szoftverhez
│
├── backend/          Szerveroldali alkalmazás (Node.js/Express)
│   ├── api/                  API útvonalak és végpontok definíciói
│   ├── middleware/           Köztes szoftverek (pl. hitelesítés, hibakezelés)
│   ├── sql/                  Adatbázis sémák, lekérdezések és kapcsolódási logika
│   ├── .env.example          Környezeti változók sablonja a helyi beállításhoz
│   ├── server.js             Az alkalmazás belépési pontja és szerver konfiguráció
│   ├── package.json          Backend függőségek és szkriptek
│   ├── utils.js              Általános segédfüggvények
│   ├── data_test.js          Bejövő adatok formai és tartalmi ellenőrzése
│   └── *.test.js             Unit és integrációs tesztfájlok
│
└── frontend/         Kliensoldali alkalmazás (HTML/JS/CSS)
    ├── html/                 Statikus HTML oldalak (belépési pontok)
    ├── css/                  Moduláris stíluslapok (oldalanként bontva)
    ├── javascript/           Kliensoldali üzleti logika és DOM manipuláció
    │   └── libs/                     Külső kliensoldali könyvtárak (pl. Sortable.js)
    └── img/                  Statikus médiatartalmak (ikonok, hátterek)
        └── allatos_profilkepek/      Felhasználói avatar készlet
```

1. Projekt struktúra

2.3 Adatbázis leírása



2. Az adatbázis szerkezete

2.3.1 Felhasználók és Adminok

users – Felhasználók táblája Ez a központi tábla, itt tárolódik minden profil alapadata.

- id: Elsődleges kulcs, automatikusan növekvő azonosító.
  - Típus: INT | Kötelező: Igen (NOT NULL) | Alapértelmezett: Nincs (AUTO_INCREMENT)
- username: Egyedi felhasználónév.
  - Típus: VARCHAR(100) (max. 100 karakter) | Kötelező: Igen (NOT NULL), Egyedi (UNIQUE)
- email: Egyedi e-mail cím a belépéshez.
  - Típus: VARCHAR(255) (max. 255 karakter) | Kötelező: Igen (NOT NULL), Egyedi (UNIQUE)
- password: Titkosított (Bcrypt) jelszó-hash.
  - Típus: VARCHAR(255) (max. 255 karakter) | Kötelező: Igen (NOT NULL)
- profil_pic_url: A felhasználó profilképének elérési útja.
  - Típus: TEXT | Kötelező: Nem | Alapértelmezett: NULL
- selected_week_type: Az órarendhez használt alapértelmezett hét.
  - Típus: VARCHAR(2) (max. 2 karakter) | Kötelező: Nem | Alapértelmezett: 'A'

admins – Adminisztrátorok táblája Meghatározza, ki rendelkezik szerkesztési joggal a globális tartalmakhoz.

- id: Elsődleges kulcs, automatikusan növekvő azonosító.
  - Típus: INT | Kötelező: Igen (NOT NULL) | Alapértelmezett: Nincs (AUTO_INCREMENT)
- user_id: Külső kulcs, amely a users(id)-re mutat (CASCADE törlés).
  - Típus: INT | Kötelező: Igen (NOT NULL)

2.3.2 Tanulókártya (Flashcards)

flashcard_deck – Tanulókártya paklik

- deck_id: Elsődleges kulcs.
  - Típus: INT | Kötelező: Igen (NOT NULL) | Alapértelmezett: Nincs (AUTO_INCREMENT)
- deck_name: A pakli neve.
  - Típus: VARCHAR(200) (max. 200 karakter) | Kötelező: Nem
- create_date: A létrehozás időpontja.
  - Típus: TIMESTAMP | Kötelező: Nem
- position: A paklik sorrendje a felhasználói felületen.
  - Típus: INT | Kötelező: Igen (NOT NULL)
- public: Logikai érték, látható-e mások számára.
  - Típus: BOOLEAN | Kötelező: Nem | Alapértelmezett: FALSE
- share_code: Egyedi 8 karakteres kód a pakli megosztásához.
  - Típus: VARCHAR(8) (max. 8 karakter) | Kötelező: Nem, Egyedi (UNIQUE) | Alapértelmezett: NULL
- user_id: Külső kulcs, a pakli tulajdonosa (users(id), CASCADE törlés).
  - Típus: INT | Kötelező: Igen (NOT NULL)

flashcard_card – Egyedi kártyák

- card_id: Elsődleges kulcs.
  - : INT | Kötelező: Igen (NOT NULL) | Alapértelmezett: Nincs (AUTO_INCREMENT)
- deck_id: Külső kulcs a paklihoz (flashcard_deck(deck_id), CASCADE törlés).
  - Típus: INT | Kötelező: Igen (NOT NULL)
- front_text: A kártya eleje (kérdés/fogalom).
  - Típus: VARCHAR(255) (max. 255 karakter) | Kötelező: Nem

- back_text: A kártya hátulja (válasz/definíció).
  - Típus: TEXT | Kötelező: Nem
- position: A kártya helye a paklin belüli sorrendben.
  - Típus: INT | Kötelező: Igen (NOT NULL)

2.3.3 Kvíz (Quizzes)

quizzes – Kvízek főtáblája

- quiz_id: Elsődleges kulcs.
  - Típus: INT | Kötelező: Igen (NOT NULL) | Alapértelmezett: Nincs (AUTO_INCREMENT)
- user_id: Külső kulcs a kvíz készítőjéhez (users(id), CASCADE törlés).
  - Típus: INT | Kötelező: Igen (NOT NULL)
- title: A kvíz címe.
  - Típus: VARCHAR(200) (max. 200 karakter) | Kötelező: Igen (NOT NULL)
- description: Rövid leírás a kvíz tartalmáról.
  - Típus: TEXT | Kötelező: Nem | Alapértelmezett: NULL
- last_modified: Utolsó módosítás időpontja.
  - Típus: TIMESTAMP | Kötelező: Nem | Alapértelmezett: CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
- public: Logikai érték, látható-e mások számára.
  - Típus: BOOLEAN | Kötelező: Nem | Alapértelmezett: FALSE
- randomize_questions: Keverje-e össze a kérdéseket indításkor.
  - Típus: BOOLEAN | Kötelező: Nem | Alapértelmezett: FALSE
- total_points: A kvízben megszerezhető összes pontszám.
  - Típus: INT | Kötelező: Nem | Alapértelmezett: 0
- position: Megjelenítési sorrend.
  - Típus: INT | Kötelező: Igen (NOT NULL)

quiz_questions – Kvíz kérdések

- question_id: Elsődleges kulcs.
  - Típus: INT | Kötelező: Igen (NOT NULL) | Alapértelmezett: Nincs (AUTO_INCREMENT)
- quiz_id: Külső kulcs a kvízhez (quizzes(quiz_id), CASCADE törlés).
  - Típus: INT | Kötelező: Igen (NOT NULL)
- question_text: Maga a kérdés szövege.
  - Típus: TEXT | Kötelező: Igen (NOT NULL)
- question_type: A kérdés fajtája.
  - Típus: ENUM('standard', 'order', 'short', 'fill') | Kötelező: Nem | Alapértelmezett: 'standard'
- points: Mennyi pontot ér a helyes válasz.
  - Típus: INT | Kötelező: Igen (NOT NULL)
- position: Kérdések sorrendje.
  - Típus: INT | Kötelező: Igen (NOT NULL) | Alapértelmezett: 1

quiz_answers – Válaszlehetőségek

- answer_id: Elsődleges kulcs.
  - Típus: INT | Kötelező: Igen (NOT NULL) | Alapértelmezett: Nincs (AUTO_INCREMENT)
- question_id: Külső kulcs a kérdéshez (quiz_questions(question_id), CASCADE törlés).
  - Típus: INT | Kötelező: Igen (NOT NULL)
- answer_text: A válasz szövege. :
  - FIGYELEM: A quiz_answers.answer_text oszlop fill típusú kérdéseknél JSON-t tárol szövegként. Ezeket mindig JSON.parse()-szal kell feldolgozni!
  - Típus: TEXT | Kötelező: Igen (NOT NULL)
- right_answer: Logikai érték, ez-e a helyes válasz.
  - Típus: BOOLEAN | Kötelező: Igen (NOT NULL)
- points: Adott válaszért járó pont (részpontozáshoz).
  - Típus: INT | Kötelező: Igen (NOT NULL) | Alapértelmezett: 1

2.3.4 Eredmények, Megosztások és Kedvencek

quiz_submit – Kvíz beküldések

- result_id: Elsődleges kulcs.
  - Típus: INT | Kötelező: Igen (NOT NULL) | Alapértelmezett: Nincs (AUTO_INCREMENT)
- quiz_id: Melyik kvízt töltötték ki (quizzes(quiz_id), CASCADE törlés).
  - Típus: INT | Kötelező: Igen (NOT NULL)
- user_id: Ki töltötte ki (users(id), CASCADE törlés).
  - Típus: INT | Kötelező: Igen (NOT NULL)
- taken_at: A kitöltés dátuma és ideje.
  - Típus: TIMESTAMP | Kötelező: Nem | Alapértelmezett: CURRENT_TIMESTAMP
- total_points: Maximális elérhető pont a kitöltéskor.
  - Típus: INT | Kötelező: Igen (NOT NULL) | Alapértelmezett: 0
- earned_points: A felhasználó által elért pontszám.
  - Típus: INT | Kötelező: Igen (NOT NULL) | Alapértelmezett: 0

quiz_results – Részletes válaszok

- result_id: Kapcsolat a beküldéshez (quiz_submit(result_id), CASCADE törlés).
  - Típus: INT | Kötelező: Igen (NOT NULL)
- question_id: Melyik kérdésre válaszolt (quiz_questions(question_id), CASCADE törlés).
  - Típus: INT | Kötelező: Igen (NOT NULL)
- answer_text: Mit válaszolt a felhasználó.
  - FIGYELEM: A quiz_results.answer_text JSON-t tárol szövegként. Ezeket mindig JSON.parse()-szal kell feldolgozni!
  - Típus: TEXT | Kötelező: Igen (NOT NULL)
- points_earned: Hány pontot kapott erre az egy válaszra.
  - Típus: INT | Kötelező: Igen (NOT NULL) | Alapértelmezett: 0

quiz_share – Kvíz megosztás adatok

- result_id: Kapcsolat a kitöltés eredményéhez (quiz_submit(result_id), CASCADE törlés).
  - Típus: INT | Kötelező: Igen (NOT NULL)
- quiz_id: Kapcsolat a kvízhez (quizzes(quiz_id), CASCADE törlés).
  - Típus: INT | Kötelező: Igen (NOT NULL)
- user_id: Kapcsolat a felhasználóhoz (users(id), CASCADE törlés).
  - Típus: INT | Kötelező: Igen (NOT NULL)

- position: Megjelenítési sorrend.
    - Típus: INT | Kötelező: Igen (NOT NULL)

user_favorites – Kedvencek

- id: Elsődleges kulcs.
    - Típus: INT | Kötelező: Igen (NOT NULL) | Alapértelmezett: Nincs (AUTO_INCREMENT)
- user_id: Ki jelölte meg kedvencnek (users(id), CASCADE törlés).
    - Típus: INT | Kötelező: Igen (NOT NULL)
- item_type: Milyen típusú elemet mentett el.
    - Típus: ENUM('quiz', 'flashcard') | Kötelező: Igen (NOT NULL)
- item_id: A mentett elem azonosítója.
    - Típus: INT | Kötelező: Igen (NOT NULL)
- created_at: A kedvencként való mentés ideje.
    - Típus: TIMESTAMP | Kötelező: Nem | Alapértelmezett: CURRENT_TIMESTAMP
- Extra megkötés: UNIQUE KEY (user_id, item_type, item_id) (Egy adott elemet csak egyszer lehet kedvencnek jelölni).

2.3.5 Szervezés és Időmenedzsment

todo_tasks – Teendők

- id: Elsődleges kulcs.
    - Típus: INT | Kötelező: Igen (NOT NULL) | Alapértelmezett: Nincs (AUTO_INCREMENT)
- user_id: A feladat tulajdonosa (users(id), CASCADE törlés).
    - Típus: INT | Kötelező: Igen (NOT NULL)
- task_name: A feladat rövid neve.
    - Típus: VARCHAR(255) (max. 255 karakter) | Kötelező: Igen (NOT NULL)
- task_description: Részletesebb leírás.
    - Típus: TEXT | Kötelező: Nem
- importance: Prioritás foka.
    - Típus: ENUM('high', 'medium', 'low') | Kötelező: Igen (NOT NULL)
- is_completed: Kész van-e a feladat.
    - Típus: BOOLEAN | Kötelező: Igen (NOT NULL) | Alapértelmezett: FALSE
- created_at: A teendő rögzítésének ideje.
    - Típus: TIMESTAMP | Kötelező: Nem | Alapértelmezett: CURRENT_TIMESTAMP

events – Események/Naptár

- event_id: Elsődleges kulcs.
  - Típus: INT | Kötelező: Igen (NOT NULL) | Alapértelmezett: Nincs (AUTO_INCREMENT)
- user_id: Az esemény tulajdonosa (users(id), CASCADE törlés).
  - Típus: INT | Kötelező: Igen (NOT NULL)
- event_date: Az esemény napja.
  - Típus: DATE | Kötelező: Igen (NOT NULL)
- title: Az esemény neve.
  - Típus: VARCHAR(255) (max. 255 karakter) | Kötelező: Igen (NOT NULL)
- description: Részletes leírás.
  - Típus: TEXT | Kötelező: Nem

timetable – Órarend

- event_id: Elsődleges kulcs.
  - Típus: INT | Kötelező: Igen (NOT NULL) | Alapértelmezett: Nincs (AUTO_INCREMENT)
- user_id: Az óra tulajdonosa (users(id), CASCADE törlés).
  - Típus: INT | Kötelező: Igen (NOT NULL)
- day: A hét napja számmal.
  - Típus: INT | Kötelező: Igen (NOT NULL)
- start_time: Tanóra kezdete.
  - Típus: TIME | Kötelező: Igen (NOT NULL)
- end_time: Tanóra vége.
  - Típus: TIME | Kötelező: Igen (NOT NULL)
- subject: Tantárgy neve.
  - Típus: VARCHAR(100) (max. 100 karakter) | Kötelező: Nem
- location: Terem vagy helyszín.
  - Típus: VARCHAR(50) (max. 50 karakter) | Kötelező: Nem
- week_type: Melyik héten van az óra.
  - Típus: VARCHAR(2) (max. 2 karakter) | Kötelező: Nem

2.4 Megvalósítások

A BeeReady három html fájlt tartalmaz:

- index.html: A bejelentkezést, a regisztrációt és az elfelejtett jelszó elemeket tartalmazza.
- main.html: A sikeresen regisztrált vagy bejelentkezett felhasználó ide kerül átirányításra. Ez tehát a főoldal, amely az összes fő elemet tartalmazza: fő oldal, tanulókártyák, órarend, kvízek, hive közösségi felület és a profil szerkesztését.
- admin.html: Az admin jogosultsággal rendelkező felhasználók ide kerülnek átirányításra. Itt tudják kezelni a felhasználók és az adminok adatait: felhasználó nevüket, email címüket, profil képüket (természetes felhasználó esetén) és jelszavakat.

A frontenden általánosságban érvényesül a szabály miszerint, a main.html mainelem azaz a fő elemei két fájlból tevődnek össze, ezek az azonos névvel ellátott javascript és css fájlok. A bejelentkezett felhasználó a main.html kezelőfelületet használja. Abban az összes fő elem megtalálható, amelyeket a .dnone osztály rejtve tart egészen addig, ameddig a felhasználó a hozzá tartozó navigációs menü elemére nem kattint.

Egy általános API kérés folyamata:

3. Általános API kérés folyamata

2.4.1 Bejelentkezés és Regisztráció

Az alkalmazás egyetlen publikus felülete az index.html, amely egy dinamikus, kártya-alapú rendszerben egyesíti a bejelentkezési, regisztrációs és jelszó-emlékeztető funkciókat. A teljes kezelőfelület egyetlen .card CSS osztályú konténeren belül helyezkedik el, amely három, logikailag elkülönített nézetet foglal magában: az alapértelmezetten látható bejelentkezési űrlapot (#logdiv), a plusz gombra kattintva megjelenő regisztrációs felületet (#regdiv), valamint az elfelejtett jelszó linkhez tartozó #forget_password_div szekciót.

A folyamatos felhasználói élmény érdekében az oldal újratöltés helyett egy finomhangolt, kétfázisú CSS fade-out/fade-in animációval vált a nézetek között. A váltás során a rendszer először nullára csökkenti az aktuális kártya áttetszőségét, majd a tartalom tényleges cseréjét a DOM-ban egy 200 ms-os késleltetéssel (setTimeout) hajtja végre. Ez az időzítés pontosan megegyezik az animáció időtartamával, így az új nézet megjelenítése (fade-in) csak akkor kezdődik el, amikor a felület már teljesen láthatatlan, tökéletesen elfedve a váltást a felhasználó elől.

2.4.1.1 Avatár-választó modal

A BeeReady regisztráció során saját kép feltöltése helyett kilenc előre definiált állatos avatar közül lehet választani egy modal ablakban, amely a profilkép-előnézetre kattintva hívható elő. A modal láthatóságát a már meglévő .dnone osztály dinamikus hozzáadásával és elvételével, elrendezését pedig display: flex segítségével kezeljük. Az .avatar_option osztályú képek egyikére kattintva az előnézet forrása azonnal frissül, a backend felé pedig végül csak a fájlnév továbbítódik, amit a szerver egy dedikált mappából szolgál ki.

2.4.1.2 Regisztrációs form – beküldés és API hívás

A regisztrációs űrlap elküldésekor a submit esemény alapértelmezett működését az e.preventDefault() gátolja meg, így az adatok gyűjtése és az /api/registration végpontra történő továbbítása aszinkron módon zajlik. A profilkép kezelésénél a rendszer a .split('/') hívással tisztítja meg az adatokat: mivel a böngésző az src attribútumot automatikusan abszolút URL-lé egészíti ki, ebből csak a tényleges fájlnevet emeljük ki, hogy a backendnek már a megfelelő relatív útvonalat adhassuk át.

2.4.1.3 Bejelentkezési form – JWT és az Emlékezz rám opció

A bejelentkezési form három adatot küld a backendnek: e-mail, jelszó és egy stay logikai érték. Ez a stay érték a bejelentkezési form egy checkbox eleméből jön. Ha a felhasználó bejelöli az Emlékezz rám opciót (stay === true), a backend 7 napig érvényes JWT tokent állít ki. Ha nem jelöli be, az alapértelmezett érvényességi idő 1 óra.

2.4.2 Kettős Validáció

Az adatok helyességének ellenőrzésére, a frontend és backend oldalon is alkalmazunk validáló metódusokat megelőzve ezzel a hibás formátumú adatok elmentését.

Frontenden és a Backenden működő ellenőrző metódus neve a lengthtest(input, min, max), mindkét metódus három paramétert vár: adat amit tesztelni szeretnénk, a minimum és a maximum hosszúsága. Amennyiben az adat nem felel meg a követelményeknek a metódus megszakítja a futást, és az alertell(text, time) segítségével jelez a felhasználónak a hibáról. Amennyiben a hibás adat eljutna a backendhez a backendben megtalálható lengthtest() validáló metódus állítaná meg az aktuális metódus futását és hibát dobna, amit aztán az apiFetch()/index_apiFetc() metódusok kezelnének.

Az adatok helyességének ellenőrzésére több metódust is megvalósítottunk az eltérő formátum ellenőrzések érdekében, ilyen például az órarendhez használt timetest(start, end) metódus amely azt ellenőrzi, hogy egy esemény befejező időpontja ne lehessen hamarabb, mint a kezdeti, illetve ellenőrzi, hogy megfelelő formátumban legyen.

2.4.3 Központi megvalósítások frontenden

A frontenden az összes html fájlra egyaránt érvényes metódusokat a main.js fájlban valósítottuk meg. Ide tartoznak például a hibakezelések, ellenőrzések, random egész szám generátor, vagy a randomizált lista elkészítése metódusok.

2.4.3.1 Központosított response kezelés

Az index_apiFetch és az apiFetch metódusok felelnek a végpontokról érkező válaszok kezeléséért. Az index_apiFetch az index.html-hez tartozó végpontokon kapott hibákat kezeli, míg az apiFetch a main.html és az admin.html-ért felelős. A két metódusra a megegyező hibakódú azonban eltérő általános üzenetek okán volt szükség.

Feladataik:

- Hálózati kérések szabványosítása: Egységes interfészt biztosítanak a fetch hívásokhoz, kezelve az URL-eket és az opciókat (method, headers, body).
- Biztonságos JSON formátum kezelés: Felelnek a szerver válaszának feldolgozásáért. Beépített hibakezelőjük (.catch(() => ({}))) megakadályozza az alkalmazás összeomlását, ha a szerver nem valid JSON-t küld vissza.
- Központosított hiba-visszajelzés: Automatikusan kezelik a felhasználói értesítéseket az alertell függvényen keresztül, így nem kell minden egyes hívásnál külön megírni az üzeneteket megakadályozva ezzel az ismétlődést. Amennyiben a server a response-ban true

értékű write objektumot küld a kliensnek, a hibakezelő kiírja a kapott üzenetet az alertell metódus segítségével

- Alacsony szintű hálózati hibák elkapása: Felelnek a kliensoldali hibák (pl. megszakadt internetkapcsolat) felismeréséért és naplózásáért.

2.4.3.2 Értesítés megjelenítése az alertell(text, time) metódus

Az alertell feladata, hogy a beépített alert() metódus szerepét töltse be formázott és dinamikus módon. Két paraméter adható neki, az első a megjeleníteni kívánt szöveg, míg a második a szöveg megjelenésének időtartama.

2.4.3.3 Kijelentkeztetési modal

Sikeres bejelentkezés, illetve regisztráció után a DOM elemeinek betöltését követően meghívásra kerül, amely a /api/get_expire_time végponton keresztül megkapja a tokenből kiolvasott fennmaradt érvényességi időt. A metódus paraméterül kaphat egy logiokai értéket amely azt állítja, hogy milyen típusú modalt kell létrehoznia abban az esetben ha a lejárati idő 60 másodperc vagy az alá csökken. Erre azért van szükség, mivel az admin.html bootstrap alapú megjelenést használ, így oda bootstrap modalt jelenít meg.

Annak érdekében, hogy egy munkamenetben csak egyszer jelenjen meg a figyelmeztető ablak, a böngésző sessionstorage tárában létrehoz egy változót amely tárolja a megjelenést logikai érték formájában.

Ha nem szeretnénk meghosszabbítani az idő vagy nem választunk opciót akkor 5 másodperctől elkezd figyelmeztető üzeneteket megjeleníteni az alertell() metódus segítségével (visszaszámol), majd a lejáratkor automatikusan kijelentkeztet.

Hosszabbítás esetén meghívja a get_extend_time() metódust amely az azonos nevű végponton keresztül kérést intéz a servernek egy új token kiállítására. Sikeres válasz esetén lecseréli a meglévőt a localStorage-ben és újra indítja az időzítőt.

2.4.3.4 Munkamenet kezelés és lejárat

A start_logout_timer() függvény a GET /api/get_expire_time végpontról lekéri a JWT-token hátralévő érvényességi idejét másodpercekben. Ezután setInterval-lal másodpercenként csökkenti a számlálót. Ha az idő eléri a 60 másodpercet és a figyelmeztetés még nem jelent meg (sessionStorage modal_showed !== „true"), megjelenik egy megerősítő modal a meghosszabbítási lehetőséggel. Ha az idő 5 másodperc alá csökken, alertell() üzenet jelenik meg a képernyőn. Ha az idő eléri a nullát, a logout() függvény törli a JWT-tokent a localStorage-ból és visszairányítja a felhasználót a bejelentkezési oldalra.

A munkamenet meghosszabbítása az extend_time() függvénnyel történik, amely a GET /api/get_extended_time végpontot hívja meg. A szerver visszaad egy friss JWT-tokent, amelyet a függvény eltárol a localStorage-ban, és újraindítja a lejárati időzítőt (start_logout_timer()).

2.4.4 Sortable.js könyvtár

A Sortable.js egy könnyűsúlyú, keretrendszer-független eszköz, amely interaktívvá teszi a listákat és konténereket. Segítségével a felhasználók egérrel vagy érintéssel (touch) mozgathatják az elemeket, átrendezve azok sorrendjét.

Főbb jellemzők, amik megjelennek a projektben:

- Natív támogatás: Nem igényel jQuery-t, így gyors és alacsony az erőforrásigénye.
- Sima animációk: Beépített animation tulajdonsággal rendelkezik, amely látványosabbá teszi az elemek helycseréjét (például a kártyák elcsúszását).
- Érintőképernyő támogatás: Mobileszközökön is zökkenőmentesen működik.
- Csoportosítás (Groups): Lehetővé teszi elemek áthúzását egyik listából a másikba.

2.4.5 Főoldal

A főoldal (main.html #fooldal) a bejelentkezés utáni alapértelmezett nézet, amely egységes, produktivitásközpontú munkaterületet biztosít a felhasználónak. A megjelenített widgetek a mainpage.js és a mainpage.css fájlokban vannak implementálva. Az oldal négy fő modulból áll: teendőlista (ToDo), időzítő, naptár, valamint egy statisztikai áttekintő.

2.4.5.1 Teendőlista (ToDo)

A teendőlista modul (#tasks) lehetővé teszi a felhasználó számára saját feladatainak kezelését. A feladatok létrehozásához egy űrlap áll rendelkezésre, amelyen megadható a feladat neve, leírása, és fontossági szintje (Magas, Közepes, Alacsony). A mentés a submitTask() függvényen keresztül a POST /api/taskadd végpontra küld kérést, amely JWT-token alapú hitelesítést igényel.

A feladatok listázása a loadTasks() függvénnyel történik (GET /api/gettasks), amelyet az oldal betöltésekor a DOMContentLoaded esemény automatikusan meghív. Minden feladatot a createTaskElement() függvény épít fel dinamikusan a DOM-ban: megjeleníti a feladat nevét, leírását, fontossági szintjét, valamint a lehetséges műveleteket. Az aktív (nem kész) feladatoknál Szerkesztés és Kész gomb jelenik meg, a kész feladatoknál pedig Vissza és Törlés.

A feladat szerkesztése helyi szerkesztési módban (enableEditMode()) történik: a cím contentEditable-ra vált, a leírás textarea szerkeszthetővé válik. A mentés a saveTask() függvénnyel a POST /api/updatetask végpontra küld kérést. A Kész gomb a markTaskDone() (POST /api/marktaskdone), a Vissza gomb a restoreTask() (POST /api/restoretask), a Törlés gomb pedig a deleteTask() (DELETE /api/deletetask) végpontot hívja meg.

2.4.5.2 Időzítő

Az időzítő (.timer_div) widget három üzemmódban működik, amelyek között egy legördülő listával (timer_select) lehet váltani. Stopper módban a másodpercek felfelé számlálódnak. Pomodoro módban 25 perces visszaszámláló indul, amely lejártakor alertell() értesítést küld. Egyéni időzítő módban a felhasználó percben és másodpercben adhatja meg az időt (pl. 05:30 formátumban); az input csak számokat és kettőspontot fogad el, és automatikusan illeszti be a kettőspontot a kétjegyű percérték után.

A vezérlőgombok (Indítás, Leállítás, Visszaállítás) állapota a toggleButtons() függvény kezeli: a Leállítás csak futás közben aktív, a Visszaállítás csak akkor, ha az időzítő el van indítva vagy el van térítve az alapállapottól. A futó időzítő közben a totalSecondsSpent változó minden másodpercben növekszik, és az updateStudyTime() függvény frissíti az aktuális munkamenetben eltöltött tanulási időt a #total_time_display elemben (óra/perc/másodperc formátumban).

2.4.5.3 Naptár

A naptár widget (#calendar szekció) havi nézetben jeleníti meg a napokat. Az update_Cal() függvény minden hónapváltáskor lekéri az adott hónap eseményeit a POST /api/get_calendar_events végpontról (év és hónap paraméterekkel), majd dinamikusan felépíti a naptárrács celláit. A mai nap kiemelve jelenik meg (active CSS osztály), az eseményes napok pedig egy kis ponttal (#event-dot) vannak jelölve; ezekre rámutatva tooltip mutatja az esemény nevét és leírását.

Egy napra kattintva megnyílik az eseménymodal (openEventModal()), amely megjeleníti az adott nap eseményeit, és lehetővé teszi új esemény hozzáadását (cím és opcionális leírás megadásával). Az esemény mentése a POST /api/insert_calendar_event végpontra küld kérést. Az eseményeket törölni lehet a modal Törlés gombjával (POST /api/delete_calendar_event). A hónapok közötti navigálást a ‹ és › gombok biztosítják, amelyek a current_date objektum hónapját módosítják, majd újra meghívják az update_Cal() függvényt.

2.4.5.4 Statisztika

A statisztika widget (#statistics szekció) a teendőlista aktuális állapotát vizualizálja egy körkörös diagramon (.circle CSS elem). Az updateStatisticsChart() függvény minden loadTasks() hívás után automatikusan lefut: megszámolja a Magas, Közepes, Alacsony fontosságú és a Kész feladatokat, majd CSS custom property-ket (--high-p, --medium-p, --low-p, --done-p) állít be a conic-gradient alapú diagramon. A jelmagyarázat (#caption) dinamikusan jeleníti meg az összes feladatszámot kategóriánként. Ha egyetlen feladat sincs, a diagram 100%-ban a „kész” állapotot mutatja.

2.4.6 Tanulókártyák

A tanulókártyák modul a #tanulokartyak szekció alatt helyezkedik el a main.html-ben. A logika a flashcards.js, a stílus a flashcards.css fájlban található. A modul három fő nézetből áll: pakli lista, kártya lista, és játék nézet.

2.4.6.1 Pakli lista nézet (#decks)

Az oldal betöltésekor ez az alapértelmezett nézet. Tartalmaz egy űrlapot (#flashcard_form) új pakli létrehozásához, illetve egy konténert (#deckList) a dinamikusan renderelt paklikártyáknak. A paklielemeket a build_deck() függvény generálja JavaScriptből.

Az input két funkciót lát el. Megadható neki egy új pakli neve, illetve megadható neki egy egyedi ’#’-el kezdődő 8 karakterből álló megosztási kód. A form submit eseményére ellenőrződik az adatok helyessége, illetve hogy egy azonosítót kapott-e. Amennyiben igen az adatbázisban duplikálásra kerül a pakli a felhasználó számára azzal a különbséggel, hogy a deck az auto increment miatt új id-t kap, illetve a user_id a munkamenetet kezelő felhasználóé lesz.

2.4.6.2 Kártya lista nézet (#cards)

Egy pakli megnyitásakor (deck_open()) a #decks div elrejtődik, a #cards div megjelenik. Tartalmazza a pakli nevét (#currentDeckName), a fejléc-gombokat (Indítás, Szerkesztés, Kártya hozzáadása, Vissza), a kártya konténert (#cardList), és a játék konténert (#flashcardGameContainer). A kártyaelemeket a build_card() függvény generálja a DOM segítségével.

2.4.6.3 Játék nézet (#flashcardGameContainer)

A kártya lista nézetbe integrálva jelenik meg. Egy 3D flip animációval rendelkező kártyából (#flashcard) és két navigációs gombból (#prevCardBtn, #nextCardBtn) áll. A .dnone_on_start CSS osztállyal jelölt elemek (pl. kártyalista, fejléc-gombok) játék közben automatikusan elrejtődnek.
Miután kiválasztottuk az indítási módot (sorrendben, illetve random), az algoritmus betölti a kártyák elejét és hátoldalát a localstorage-ba, illetve, hogy hányadik kártyánál tart. Random mód esetén nem a Sortable könyvtár szerinti sorrend alapján fog elindulni, hanem a lista hossza alapján generál számokat amely alapján betölti majd a servertől kapott kártyákat.

2.4.7 Órarend

Az órarend modul lehetővé teszi, hogy a felhasználó heti bontásban rögzítse az órarendjét. Támogatja a heti/páros/páratlan hetes beosztást, és mobilon napválasztós nézetbe vált. A frontend logika a timetable.js-ben, a stílus a timetable.css-ben, a backend routeok az api.js-ben, az adatbázis-lekérdezések a querys.js-ben találhatók.

2.4.7.1 Nézet és DOM struktúra

Az órarend a #timetable szekció alatt helyezkedik el. Az elrendezés egy vízszintes flex konténer (.timetable-flex-container), amely 5 oszlopból áll (Hétfő–Péntek), mindegyik egy .day-column div. Az eseménykártyák (.event-card) ezekbe az oszlopokba kerülnek dinamikusan. Minden naposzlop egy #day_0 – #day_4 id-val rendelkezik, ezekre kerülnek az események a build_event() függvény által.

Mobilon (≤1024px) az összes .day-column el van rejtve, és egy .mobile-day-selector gombsor jelenik meg, amellyel a felhasználó kiválaszt egy napot. A kiválasztott nap .active-day osztályt kap és display: block lesz. A napi gombsorra (showDay()) kattintva csak az adott nap oszlopa válik láthatóvá.

2.4.7.2 Hétválasztó

A fejlécben lévő #weekSelector select elem értéke („A” / „B”) alapján töltődnek be az események. Értékváltáskor a change_selected_week API végpont elmenti a felhasználó preferenciáját, majd load_events() frissíti a nézetet. Betöltéskor a szerver visszaküldi az elmentett értéket, és a selector automatikusan erre áll be.

Az elmentett értéket a users táblában tároljuk a selected_week_type mezőben amely alapértelmezés szerint A értéket vesz fel tehát az első megváltoztatásig automatikusan az „A” hetet mutatja.

2.4.7.3 Az aktuális nap megjelölése

Az órarend modulban a jelenlegi nap egy border-t kap kiemeleséként az today_highlight osztály segítségével. A modul hétfőtől indexel és 0-val kezd ezért egy plusz műveletet kell elvégezni a JavaScript beépített Date osztályának getDay() metódusának hívásakor, mivel a getDay() metódus Vasárnaptól kezdi az indexelést 0-val ezért a visszakapott értékből levonunk 1-et.

Amennyiben negatív értéket kapunk vagy olyan értéket amely nagyobb, mint 4 akkor nem jelölünk meg napot kiemelésre. Mobilnézetben amikor az órarendmodult nyitjuk meg automatikusan először az aktuális nap nyílik meg, mivel betöltéskor annak a gombnak adjuk meg az active-day osztályt.

2.4.8 Kvíz

A kvíz modul felelős, azért, hogy a felhasználó ellenőrizni tudja tudását, illetve gyakorolni tudjon az egyes témakörökből. Lehetővé teszi a felhasználók számára, hogy saját kvízeket hozzanak létre, szerkesszenek, kitöltsenek és az eredményeiket megtekinthessék. A modul négy kérdéstípust támogat, és képes a mások által nyilvánossá tett kvízek kitöltésére is.
Felépítés szempontjából négy fő elemből áll:

- A saját illetve a nem saját tulajdonú kvízek listázása
- A kvízek létrehozása
- A kvízek szerkesztése
- A kvízek kitöltése
- A kitöltés eredményének megtekintése

2.4.8.1 A saját illetve a nem saját tulajdonú kvízek listázása

Amikor a felhasználó a navigációban a Kviz menüpontra kattint, a load_quizzes() függvény hívódik meg. Ennek menete:

- Ellenőrzi, hogy aktív kvízkitöltés zajlik-e (sessionStorage.quiz_started)
- Ha nem, visszaállítja a nézetet a listanézetre és törli a korábbi tartalmakat
- GET /api/getquizzes: betölti a saját kvízeket
- GET /api/getforeignquizzes: betölti a mentett idegen kvízeket
- A kapott adatokból build_quiz() segítségével DOM-elemeket épít
- A Sortable.js könyvtárral drag-and-drop átrendezést tesz lehetővé mindkét listán

Az idegen kvízek a checkbox bepipálásával válnak elérhetővé a .dnone osztály kivételével, illetve a saját tulajdonú kvízek konténeréhez hozzáadásra kerül.

A build_quiz() függvény dinamikusan hozza létre az egyes kvízkártyákat a következő elemekkel:

- Cím, leírás, kérdések száma, létrehozás dátuma, szerző
- Drag handle a sorrendező fogóponthoz
- Gombok: Indítás, Szerkesztés (csak saját quiz esetén), Eredmények, Törlés

2.4.8.2 A kvízek létrehozása

A kvíz lista felett megtalálható plusz jelre kattintva meghívásra kerül a showcreatequiz() metódus, amely a .dnone osztály segítségével eltűnteti a listákat és megjeleníti a quiz létrehozásához szükséges modulokat, illetve inicializálja a Sortable könyvtárat a kérdések konténerére. A szerkesztő egy HTML form (#quizCreateForm), amelybe dinamikusan kerülnek be a kérdésblokkok.

2.4.8.2.1 Kérdéstípus-választó modál – showQuizTypeSelector()

Egy modális ablak jelenik meg négy gombbal (Feleletválasztós, Rövid válasz, Sorba rendező, Kitöltő), amely az adott kérdéstípus hozzáadó függvényét hívja.

Minden kérdéstípushoz dedikált függvény tartozik:

- addNewStandardQuestionBlock() - feleletválasztós, checkbox + szöveg + pontszám mezők
- addNewShortAnswerQuestionBlock() - rövid válasz, egy szöveges beviteli mezővel
- addNewOrderQuestionBlock() - sorba rendezős, Sortable.js-el belső rendezéssel
- addNewFillQuestionBlock() - kitöltős, speciális {szó; pont} formátumú textarea

Minden blokk a createBaseQuestionBlock() alapfüggvényre épül, amely tartalmazza a drag handle-t, a kérdésszöveg-mezőt, a törlésgombot és a pontozóelemet (kivéve a kitöltős és a sorba rendezős feladattípusnál). A sorba rendező típusnál hozzáadásra kerül egy egyéni pontozás label elemmel ellátott checkbox, ez lehetőséget biztosít arra, hogy a felhasználó egyénileg adja meg, hogy bontani szeretné-e a pontokat vagy sem.

A feleletválasztós, a rövid válasz és a sorba rendezős feladattípushoz tartozik egy-egy egyedi függvény, amely a kérdéstípusnak megfelelő extra mezőt adja hozzá a kérdés blokkhoz. Paraméterül kapják az egyes kérdés blokkok azonosítóját amelyhez hozzáadásra kerül az extra mező.

- Feleletválasztás esetén addStandardAnswerToBlock(id, question=null) - egy új válaszlehetőség ad hozzá (checkbox, input mező, pontszám beállító, és a válaszlehetőség törlése)
- Rövid válasz esetén addShortAnswerToBlock(question_id, answer = null) - egy új helyes válasz lehetőséget adhatunk hozzá(input, törlés gomb)
- Sorba rendező feladat esetén egy új rendezhető addOrderAnswerToBlock(question_id, answer = null)
  mezőt ad hozzá(input, törlés)

A kitöltő típusnál a kérdésblokkban egy input mező található.

2.4.8.2.2 A kvíz elmentése

A kvíz mentése gombra kattintáskor lefut a form submit eseménye amelyre addeventlistener-t állítottunk be. Az esemény bekövetkezésekor meghívásra kerül a SaveQuiz() metódus, amely meghívja a quiz_check() függvény melynek visszatérési értéke az összpontszám lesz. A quiz_check() ellenőrzi, hogy van-e legalább 1 kérdés és abban legalább 1 válasz, majd a függvény bejárja a DOM-ban a .question-card osztállyal ellátott elemeket, majd a típusnak megfelelően validálja a kérdést és a hozzá tartozó extra mezőket, illetve megszámolja a hozzá tartozó pontszámokat.

A validálás után a quiz egyéb meta adatait eltároljuk, majd mentésre kerül az adatbázisba, az adatbázis mentéséért felelős függvény visszatér a az adatbázisba beillesztett új sor azonosítójával, erre a továbbiakban az idegenkulcsok miatt lesz szükség.

Ezt követően a SaveQuiz metódus bejárja a kérdés blokkokat, kigyűjti a szükséges adatokat, illetve a pontszámukat és elmenti az adott kérdés a quiz_questions táblába a mentés függvénye szintén a beillesztett sor azonosítójával tér vissza, amelyet a kérdések mentésekor használunk fel.
Amennyiben a sorba rendező típusnál nincs bepipálva az egyéni pontozás a kérdés pontszáma az adatbázisban nulla lesz, egyébként pedig a beállított pontszám, ennek a célja, hogy betöltéskor az algoritmus el tudja, dönteni, hogyan jelenjen meg, illetve hogy hogyan kell értékelni.

Ezt követően a blokkban található kérdések bejárása és elmentése következik típusnak megfelelően. A kérdés pontszáma feleletválasztós, illetve a rövid válasz típusnál a pont elemekben megadott érték. A sorba rendező típus esetében, minden válasz 1 pont lesz és helyesnek lesz elmentve, úgy mint a rövid válasznál.

2.4.8.2.3 A kitöltő feladattípus mentése

A kitöltő feladattípusnál meghívásra kerül a fill_get_data() függvény, amely a szövegből kinyert kitöltendő szavakat és a hozzájuk tartozó pontokat.

A fill_get_data() metódus bejárja a kapott stringet ’{’ speciális karaktert keresve, amikor talál egyet igazra állítja a found nevű logikai változót és innentől kezdve a záró ’}’ karaktert keresi, amennyiben talál, ellenőrzi, hogy a két speciális karakter között milyen hosszúságú string található, ha üres eldobja azt máskülönben viszont megtartja és odaadja a pont kereső fill_search_points(word) metódusnak. Nem lezárt kitöltendő rész esetén hibát dob és tájékoztatja a felhasználót.

A fill_search_point(word) függvényben az eldöntés tétel alkalmazásával ellenőrizzük, hogy található-e a szövegben pontszám megadására alkalmas karakter, a ciklus a string végéről indul a megadási forma okán.
Ha talált formátumnak megfelelő karaktert, eltávolítja a felesleges szóközöket és ellenőrzi, hogy szám-e, amennyiben számot talál, konvertálja int típussá és egy objektumban visszaadja azt, máskülönben az alapértelmezett 1 pontot állítja be a kitöltendő résznek. Egy kitöltős feladattípus adatszerkezete az adatbázis answer_text táblájában: {"words":["a szöveg ami a felhasználónak meg kell adnia"],"text":"Ez az alap szöveg és ide kell majd beírni a választ:{} feladat.","points":[5]}

2.4.8.3 A kvízek szerkesztése

A szerkesztés betöltése és egyes eljárásai megegyeznek a létrehozásban leírt eljárásokhoz. A szerkesztésgombra kattintáskor a quiz_edit_user(quiz) megkapja paraméterül a load_quizzes() metódus betöltésekor létrehozott kvízhez tartozó objektumot, amely a quiz adatait tárolja, mint pl.:(leírás, cím, id…) a quiz_edit_user() metódus lekéri az adatbázisból a kvízhez tartozó kérdéseket és válaszokat, majd a típusuk alapján meghívja a létrehozáskor használt kvíz kérdés blokk kreáló függvényeket, amelyek ezúttal paraméterül kapják az alapértelmezetten null értékűnek beállított kérdés adatokat és válasz adatokat az adatbázisból lekérdezett adatokból. Majd az adott kérdés típusnak megfelelően felépülnek a létrehozásban ismertetett módon a kérdés blokkok és a hozzájuk tartozó extra mezők.

A szerkesztés mentésekor töröljük az eddig elmentett kvízt és a fentiekben leírt módon elmentjük újként a kvízt.

2.4.8.3.1 Kitöltő típus feladat betöltése

A kitöltő típusnál, a betöltéskor a kérdés blokkban található input mező értéke a fill_give_data(ansText) függvény visszatérési értékével lesz egyenlő. Az adatbázisból kapott objektumot átalakítjuk a megfelelő formátumra majd a pontszámot, a szöveget, és a kitöltendő stringeket változókban eltároljuk. Elindítunk egy ciklust amely megkeresi a ’{’ speciális karaktert és a fill_insert() függvény segítségével beillesztjük a megfelelő formátumban a szöveget és a pontszámot.

2.4.8.4 A kvízek kitöltése

Amikor a felhasználó az "Indítás" gombra kattint, a quiz_start() függvény fut le:

- sessionStorage-ba ír (quiz_started = true), hogy lapfrissítésnél figyelmeztessen
- Lekéri a kvíz kérdéseit: GET /api/getquizquestions
- Ha randomize_questions = true, véletlenszerű indexsorrendben jeleníti meg a kérdéseket (random_array())
- feleletválasztó, kitöltő és sorba rendező típusú kérdéseknél lekéri a válaszokat is: GET /api/getquestionanswersforstart (a helyes válasz jelölése nélkül)

Betölti a kvíz meta adatait és megjeleníti a kitöltő konténert és elrejti a többi elemet.
Minden kérdéstípushoz dedikált start_addNew...QuestionBlock() függvénnyel épít DOM-elemet a létrehozáshoz hasonló módon természetesen a szerkesztési lehetőségek nélkül.

A lapról való távozáshoz figyelmeztetés kapcsolódik a warn_before_unload_with_active_quiz() eseménykezelőn keresztül (beforeunload esemény).

2.4.8.4.1 Kvíz beküldése

A form beküldésekor a submitQuiz() összegyűjti a felhasználó válaszait típusonként eltérő módon:

- standard: a checkbox-ok állapotát tömbbe gyűjti (true/false tömbként)
- fill: a fill-input mezők értékeit egyenként veszi
- short: az egyetlen szövegmező értékét veszi
- order: a válasz-sorok data-id attribútumait (answer_id) a DOM sorrendjében veszi

Az összeállított adattömb POST /api/savequizresult végpontra kerül. Ha idegen kvíz volt (data-submit-type='foreign'), ezután POST /api/saveForeignquiz is meghívódik.

2.4.8.4.2 A kitöltés kiértékelése

Miután a frontenden a submitQuiz metódus összegyűjtötte a felhasználó által adott válaszokat és elküldi azokat a backend-nek a POST /api/savequizresult végponton keresztül, ahol egy ciklus bejárja a kapott tömböt melynek az első két paramétere a quiz_id és az összpontszám, majd ezután a kinyert kérdések az alábbi adatszerkezetben kerülnek kérdésenként a tömbbe: data = { question_id: question_id, question_type: question_type, answers: [] };. A küldött objektumban az answers kulcshoz tartozik az a tömb amely bejárásra kerül. Minden elemét odaadjuk az answer_validation(result_id, user, id).

Az answer_validation(result_id, user, id) függvény felelős az egyes kérdésekre adott válaszok helyességének eldöntéséért és az elért pontszám kiszámításáért. Először loadanswers() segítségével lekéri az adatbázisból az adott kérdés helyes válaszait, majd kérdéstípusonként eltérő logikával dolgozik.

2.4.8.4.2.1 order – Sorba rendezős kérdés

A feldolgozás két almódban fut attól függően, hogy az alkotó egyéni pontszámot adott-e meg (answers[0].question_points != 0).

a) Egyéni pont mód (question_points != 0) – egész vagy semmi elv:

- A teljes kérdés pontértéke = question_points (egyetlen egész szám)
- Végigiterál a helyes sorrendű válaszokon és a felhasználó által beküldött sorrendű answer_id tömbön párhuzamosan
- Ha bármely pozícióban az answer_id eltér, a points azonnal 0-ra áll – részbeni egyezés nem elegendő
- answers_text.ispartial = false jelzi, hogy nincs részpontszám
- Minden elemhez rögzítve: { answer: answer_id, correct: true/false }

b) Elemenként pontozott mód (question_points == 0) – részpontozás:

- Minden helyesen elhelyezett elem 1 pontot ér, a teljes elérhető pont = elemek száma
- Pozíciónként összehasonlítja az answer_id-kat; egyezés esetén points++
- answers_text.ispartial = true jelzi, hogy részpontozás van
- Minden elemhez rögzítve: { answer: answer_id, points: 1 vagy 0, correct: true/false }

2.4.8.4.2.2 short – Rövid válasz kérdés

- A felhasználó által beírt egyetlen szöveget (user.answers[0]) összehasonlítja az összes tárolt helyes válasszal lineáris kereséssel (while ciklus)
- Ha bármelyik tárolt helyes válasz szövegesen egyezik, a kérdés helyesnek minősül
- Helyes esetén: correct = true, points = answers[0].question_points (a kérdés teljes pontértéke)
- Helytelen esetén: correct = false, points = 0 marad
- A keresés kis- és nagybetű érzékeny, nincs normalizálás
- Rögzített adat: { answer: beirt_szoveg, correct: true/false }

2.4.8.4.2.3 fill – Kitöltős kérdés

- Az adatbázisból JSON formátumban tárolt answer_text-ből kinyeri a helyes szavak tömbjét (answer_words) és a hozzájuk tartozó pontszámokat (answer_points)
- Pozíciónként összehasonlítja a felhasználó által beírt szöveget (user.answers[i]) a helyes szóval (answer_words[i])
- Egyezés esetén points += answer_points[i] (az adott kitöltendő mezőhöz rendelt pont)
- Minden mezőhöz rögzítve: { answer: beirt_ertek, points: elert_pont, correct: true/false }
- A kitöltős kérdés is kis- és nagybetű érzékeny összehasonlítást alkalmaz

2.4.8.4.2.4 standard – Feleletválasztós kérdés

- A felhasználó válaszát boolean tömbként kapja meg (user.answers[i] = true/false, be volt-e jelölve az i-edik checkbox)
- Párhuzamosan iterál az adatbázisból betöltött answers tömbön és a user.answers tömbön
- Egyezés feltétele: answers[i].right_answer == user.answers[i] – a checkbox állapota megegyezik azzal, hogy az adott válasz helyes-e
- Ha egyezés van, points += answers[i].points (az adott válaszhoz rendelt pontszám)
- Minden válaszhoz rögzítve: { answer: checkbox_allapot, points: elert_pont, correct: true/false }
- Több helyes választós kérdésnél minden egyes válasz önállóan pontozódik

2.4.8.4.3 Közös lezárás – save_result()

Minden kérdéstípusnál a kiértékelés után az answers_text objektum JSON stringként és az elért points értékkel INSERT-elodik a quiz_results táblába a save_result() híváson keresztül. Ez az adat szolgál alapul a részletes visszanézeti nézethez és a calcquizpoints() összesítéséhez.

A calcquizpoints() metódus összeszámolja az elért pontokat és frissíti azokat a quiz_submit adatbázis táblában.

2.4.8.5 A kitöltés eredményének megtekintése

A kvíz modul lehetővé teszi a kiértékelés megtekintését, az eredmények gombra kattintva (show_quiz_result_modal(quiz, quiz_id)), amennyiben már történt kitöltés betölti egy táblázatba, melynek soraihoz kattintás esemény van rendelve. Az esemény bekövetkeztekor a load_result_details(quiz, result_id, formattedResult, earned_points) metódus betölti a kitöltéshez hasonló módon a kvíz meta adatait, kérdéseit és a felhasználó által adott válaszokat, amelyek a helyességét tároló logikai érték alapján zölden vagy pirosan fognak megjelenni.

Betöltéskor a metódus hozzáadja a felhasználó által adott válaszokat és a helyes válaszokat is, majd alapértelmezetten elrejti a megoldását és a megoldások megjelenítése gombra kattintva elrejti a .dnone osztály segítségével a felhasználó válaszait és megjeleníti a helyes válaszokat.

2.4.9 Hive

A Hive (main.html #hive) a BeeReady közösségi modulja, amelyen keresztül a felhasználók egymás publikus tanulókártyáit és kvízjeit böngészhetik, elindíthatják, illetve elmenthetik saját gyűjteményükbe. A logika a hive.js, a stílus a hive.css fájlban helyezkedik el. A modul betöltésekor a load_hive() függvény inicializálja a szűrőgombokat (HiveFilters()), a keresőmezőt (HiveSearch()), és betölti az összes publikus tartalmat (loadHiveData()).

2.4.9.1 Tartalom betöltése és megjelenítése

A loadHiveData() függvény a GET /api/hive végpontot hívja meg, három lekérdezési paraméterrel: type („all”, „flashcard” vagy „quiz”), search (keresési kifejezés) és favorites (logikai érték, csak a mentett elemek szűréséhez). A kérés JWT-tokennel hitelesített. A szerver a QnFSearch() adatbázis-függvénnyel keresi meg a feltételeknek megfelelő, publikus tartalmakat, és visszaadja az eredménylista elemeit.

A visszakapott adatokat a renderHiveCards() függvény dolgozza fel: minden elemhez egy kártyát (.hive_card) épít fel a DOM-ban. A kártya tartalmazza a típusjelzőt (KVÍZ vagy FLASHCARD badge), a készítő nevét, a tartalom rövid leírását (kvíz esetén a leírás szövegét, tanulókártya esetén a kártyaszámot és a megosztási kódot), a kedvelések számát (♥), valamint a lábléc-gombokat.

2.4.9.2 Szűrők és keresés

A Hive fejlécében négy szűrőgomb (.hive_btn_filter) helyezkedik el: Összes, Flashcardok, Kvízek, Mentett. A HiveFilters() függvény minden gombra click eseménykezelőt regisztrál, amely frissíti a currentFilters állapotobjektumot (type és favorites mezők), majd újra meghívja a loadHiveData() függvényt. Az aktív szűrőgomb active CSS osztályt kap.

A keresőmező (#hive_search_input) input eseményére a HiveSearch() függvény 500 ms-os késleltetéssel (debounce) frissíti a currentFilters.search mezőt, és újratölti a tartalmat. Ez megakadályozza, hogy minden leütésnél azonnal hálózati kérés induljon, így csökkentve a szerver terhelését és javítva a felhasználói élményt.

2.4.9.3 Lábléc-gombjai

A nem saját kártyákon megjelenik egy Mentés/Törlés gomb (.hive_btn_save), saját kártyán csak a mentés jelenik meg. Betöltéskor ellenőrizzük a kártya mentésének állapotát (is_saved_by_user === 1), Ha az adott tartalom már el van mentve a felhasználó kedvencei közé a gomb „Törlés” felirattal fog megjelenni, és saved CSS osztályt kap, ellenkező esetben „Mentés” felirattal. A gombra kattintva a SaveItem() függvény a POST /api/toggleFavorite végpontra küld kérést (item_type és item_id paraméterekkel), amely egy toggle logikával váltja a mentett állapotot. Sikeres mentés/törlés után a kedvelésszám azonnal frissül a POST /api/getfavoritecount végpont segítségével, és az alertell() értesítés visszajelzést ad a felhasználónak.

Az Indítás gomb (.hive_btn_view) csak akkor jelenik meg, ha a tartalom nem a saját felhasználóé (is_mine == 0). Rákattintva a navbar_click() függvénnyel a megfelelő modulra navigál (tanulókártyák vagy kvíz), majd közvetlenül elindítja a tartalmat: flashcard esetén deck_open(qnf.id, true), kvíz esetén quiz_start(null, qnf.id) hívással.

2.4.10 Profil kezelés

A navigációs sáv alján elhelyezett profil szekció (nav_profil osztályú elem) megnyomásával a felhasználó a profil szerkesztő felületre navigál, ahol megváltoztathatja a felhasználónevét, e-mail címét, jelszavát, és profilképét. A szekció betöltésekor a navbar_click(‘felhasznalo’, 5) függvény mellett a load_user() is meghívódik, amely az aktuális adatokat tölti be az űrlapba.

2.4.11 Navigációs sáv

A navigációs sáv (main.html nav#navbar elem) az alkalmazás bal oldalán elhelyezett, függőleges menü, amelyen keresztül a felhasználó az összes fő modul között válthat. A megjelenítési logika a navbar.js, a stílus a navbar.css fájlban van implementálva. Az oldal betöltésekor a DOMContentLoaded eseményre a start_logout_timer() elindul, a sessionStorage modal_showed kulcsa törlődik, és a load_image() meghívódik, hogy betöltse a bejelentkezett felhasználó profil képét és felhasználónevét a GET /api/edit_user végpontról.

2.4.11.1 Modulok közötti navigáció (navbar_click)

A navbar_click(id, index) függvény kezeli a modulváltást. Meghívásakor először az összes .mainelem osztályú div-re ráteszi a .dnone osztályt (elrejti), majd eltávolítja az active osztályt az összes .nav_items elemről. Ezután a kapott index alapján aktiválja a megfelelő menüpontot (nth-child szelektor), az id alapján pedig megjeleníti a kiválasztott szekciót a dnone eltávolításával. Mobilon a navbar bezárul (nav_open osztály eltávolítása) és a hamburger ikon ismét láthatóvá válik. Minden menüpont onclick eseménye a navbar_click() mellett egy modul-specifikus betöltő függvényt is meghív: load_deck() a tanulókártyáknál, load_events() az órendnél, load_quizzes() a kvíznél, load_hive() a Hive-nál, load_user() a profilnál.

2.4.11.2 Mobilnézet (hamburger menü)

Keskeny képernyőn a navigációs sáv alapértelmezés szerint rejtett. A hamburger ikon megnyomására a toggleMenu() függvény váltja a navbar nav_open CSS osztályát: ha az osztály megjelenik, a sáv kicsúszik és a hamburger ikon eltűnik (dnone), záráskor visszaáll. A navbar_click() híváskor is automatikusan bezárul a menü, így a felhasználó a modulváltás után rögtön a tartalomra fókuszálhat.

2.4.12 Admin felület

Az admin felület bootstrap segítségével került megvalósításra, az odalon található elemek bootstrap osztályokkal formázottak és bootstrap container-ben vannak elhelyezve.

Az oldal betöltésekor a loadUsers() és a loadAdmins() metódus betölti a megfelelő táblázatokba a DOM segítségével a felhasználókat és az adminokat, ahol az adataik szerkeszthetők és törölhetőek. Szerkesztéskor meghívásra kerül a saveAdmin avagy a saveUser(), amely frissíti az adatbázisban az update_user vagy az update_admin végponton keresztül a felhasználó adatait. Mentéskor a server ellenőrzi, hogy a jelszó mezőben bycript hash található-e, amennyiben nem meghívja a passwordTest(password) metódust, amely ellenőrzi, hogy megfelel e a biztonsági szabályoknak a karakterlánc, majd ha igen hash-t generál belőle elmenti az adatbázisba és visszaadja azt a response-ban annak érdekében, hogy az admin felület konzisztens legyen.

Az admin felület autentikációs módjai megegyeznek a main.html-ben ismertetettekkel, azonban kiegészül egy extra ellenőrzéssel, amely az admin jogosultságot validálja az adatbázis admin tábláján futtatott ellenőrzés segítségével. Az admin felületen, ugyanolyan időzítő indul, mint a maint.html-ben azzal a különbséggel, hogy itt bootstrap modal jelenik meg az idő lejártakor.

2.5 Backend megvalósítás

2.5.1 Központosított hibakezelés

A backend-ben az errorHandler.js globális middleware felel a hibák kezeléséért, amikor egy api hibára fut akkor a try-catch elkapja a hibát és a next() segítségével tovább dobja az errorHandler-nek amely formázottan, a kiválasztott információkat kiírja a konzolra, majd elküldi a választ a kliensnek.

2.5.2 Regisztráció és belépés végpontok

A /api/registration és /api/login végpontok előtt egy loginLimiter rate limiter fut le, amely megakadályozza, hogy valaki automatikusan sok próbálkozással törje fel a jelszavakat: 5 percen belül maximum 5 kísérletet engedélyez ugyanarról az IP-ről.

2.5.3 Jelszóvalidálás – passwordTest()

A jelszóerősség ellenőrzése a data_test.js fájlban lévő passwordTest() függvénnyel történik. A függvény egyszerre négy követelményt ellenőriz a Regex segítségével, és ha bármelyik nem teljesül, egy összesített hibaüzenetet dob, amelyben az összes sértett szabály szerepel.

2.5.4 Biztonság

A serveren DDOS túlterheléses támadások ellen a beépített express-rate-limiter-t használjuk amely 1 perces időablakban 300 kérést enged a server felé ip-címenként. A bejelentkezéshez és a regisztrációhoz loginLimiter kötött, amley a bruteforce attack elleni védelmet biztosítja

Minden végpont JSON web token ellenőrzéshez kötött, amennyiben lejárt vagy nincs megfelelő token, a metóuds kijelentkezteti a felhasználót a logout() metódus segítségével, amely törli a sessionStorage-ból az érvénytelen tokent és átirányítja a felhasználót az index.html kezdőoldalra.

3 Tesztelés és tesztdokumentáció

3.1 Hibanapló

3.1.1 ErrorHandler futási hiba

Hiba leírása

A globális hibakezelő middleware (errorHandler) nem futott le, amikor az API végpontokon hiba keletkezett. A szerver vagy nem küldött választ, vagy az Express alapértelmezett HTML hibaoldalát adta vissza a várt egyedi JSON válasz helyett.

A hiba oka

Rossz middleware sorrend. Az app.use(errorHandler) az útvonalak (app.use('/api', endpoints)) előtt lett deklarálva az app.js fájlban. Az Express "vízesés" modellje szerint a kérés fentről lefelé halad; ha a hibakezelő a hiba forrása felett van, a next(error) hívás sosem fogja elérni azt.

Javítás

Az app.use(errorHandler) sort áthelyeztem a fájl végére, az összes útvonal-definíció és statikus fájl kiszolgálása után, közvetlenül az app.listen() elé.

3.1.2 Jogosulatlan adathozzáférés

Sebezhetőség

Több végpont (pl. deletecard, updatecard, getcards) csak az elem egyedi azonosítóját (card_id, deck_id) ellenőrizte. Ennek következtében egy bejelentkezett felhasználó tetszőlegesen módosíthatta vagy törölhette más felhasználók kártyáit és paklijait, ha ismerte azok azonosítóját.

Kockázat: Kritikus.

Magánéleti adatok elvesztése vagy illetéktelen módosítása.

Megoldás

- Minden lekérdezésbe bevezetésre került a user_id alapú szűrés.
- Kártyák esetén, ahol nincs közvetlen user_id, egy JOIN segítségével ellenőrizzük a szülő pakli tulajdonosát.
- Bevezetésre került az affectedRows ellenőrzése: ha a művelet 0 sort érint, a rendszer 403-as hibát küld.

3.1.3 Kártya nélküli paklik kezelése és navigációs gomb állapot hiba

Hiba azonosítása

Komponens: flashcards.js. Érintett funkciók: flashcard_start(), flashcard_start_ordered(), flashcard_start_random() és changeCard().

Részletes leírás

- Üres pakli elindítása: A flashcard_start függvény nem ellenőrzi a kártyák számát a modal megnyitása előtt. Ennek következtében a felhasználó elindíthat egy olyan paklit is, amelyben nincs kártya, ami hibás állapotba helyezi a tanulási felületet.
- Navigációs gomb (Next) logikai hibája: Amikor egy pakli csak egy darab kártyát tartalmaz, az indításkor a flashcard_start függvény alapértelmezés szerint aktív állapotba helyezi a "Következő" (nextCardBtn) gombot. Mivel a changeCard függvény csak gombnyomásra ellenőrzi a határértékeket, az egyetlen kártyával rendelkező pakliknál a gomb kattintható marad, holott nincs hova továbblépni.

Reprodukciós lépések

- Hozzon létre egy új paklit kártyák hozzáadása nélkül.
- Kattintson az "Indítás" gombra (a modal megjelenik, és a folyamat elindul).
- Hozzon létre egy paklit pontosan 1 kártyával.
- Indítsa el a tanulást; a jobb oldali nyíl (nextCardBtn) aktív marad annak ellenére, hogy nincs következő lap.

Javasolt javítás

- A flashcard_start függvényben ellenőrizni kell a cardcount értékét; ha az 0, az indítás gomb legyen inaktív, vagy adjon hibaüzenetet.
- A flashcard_start_ordered és flashcard_start_random függvényekben a kártyák betöltése után azonnal le kell futtatni egy ellenőrzést, amely inaktiválja a nextCardBtn gombot, ha a cards.length <= 1.

3.1.4 Accessing non-existent property 'isexistscheck'

A rendszer Accessing non-existent property 'isexistscheck' figyelmeztetést dobott a szerver indításakor.

Kiinduló állapot

Az isexistscheck függvény a data_test.js fájlban volt definiálva. A querys.js importálta ezt a függvényt a data_test.js-ből, miközben a data_test.js is hivatkozott a querys.js tartalmára.

A hiba oka

Körkörös függőség jött létre. A Node.js modulkezelője nem tudta befejezni a data_test.js beolvasását, mielőtt a querys.js megpróbálta volna meghívni belőle az isexistscheck függvényt. Emiatt a függvény értéke undefined maradt az importálás pillanatában.

Megoldás (Refaktorálás)

- utils.js létrehozása: Az isexistscheck függvény átkerült a data_test.js fájlból a utils.js fájlba.
- querys.js módosítása: A függvényt most már a utils.js modulból importálja.
- Függőségi irány: utils.js → querys.js. A utils.js nem importál semmit a querys.js-ből, így a kör bezárult, a függőség lineárissá vált.

3.1.5 ToDo törlés – confirm() nem jelenik meg

Hiba leírása

A kész feladatok törlése gombra kattintva az alkalmazás a beépített böngészős confirm() dialógusablakot jelenítette meg megerősítés céljából. Egyes böngészők (különösen mobilon, illetve bizonyos biztonsági beállítások mellett) ezt az ablakot automatikusan elutasítják vagy egyáltalán nem jelenítik meg. Ilyenkor a törlés soha nem hajtódott végre, a felhasználó pedig semmilyen visszajelzést nem kapott – a gomb mintha nem működött volna.

A hiba oka

A deleteTask() függvény a confirm() hívástól tette függővé a törlési logika elindítását. Ez a megközelítés az alkalmazás többi részétől eltérő megjelenésű és viselkedésű volt, és böngészőfüggő korlátozásoknak volt kitéve.

Megoldás

A confirm() hívás helyett az alkalmazásban már meglévő, saját modal alapú megerősítési rendszert (hasonlóan a kijelentkeztetési modalhoz) alkalmaztunk. Ez egységes felhasználói élményt biztosít, és böngészőtől függetlenül megbízhatóan működik.

3.1.6 Hive - Keresőmező debounce – Felesleges hálózati kérések és globális változó kezelése

Probléma: Kontrollálatlan hálózati kérések

A Hive keresőmezője eredetileg várakozási idő nélkül működött, így minden egyes input esemény – azaz minden leütött karakter – azonnali hálózati kérést indított. Egy átlagos hosszúságú keresőkifejezés beírása során ez tucatnyi párhuzamos lekérdezést eredményezett, ami feleslegesen terhelte a szervert és a kliensoldali erőforrásokat.

A hiba oka

A HiveSearch() eseménykezelőjéből hiányzott a debounce logika. Nem volt olyan mechanizmus, amely megvárta volna a gépelés végét, és a setTimeout hívása előtt nem szerepelt clearTimeout() sem. Ennek következtében minden egyes input esemény egy újabb, párhuzamosan futó időzítőt vagy azonnali kérést generált a korábbiak leállítása nélkül.

Megoldás: A Várakoztatás megoldás

A megoldást a debounce technika szakszerű alkalmazása jelentette. A folyamat most a következőképpen működik:

1. Változó deklarálása: A timeout változót a megfelelő hatókörben (scope), let kulcsszóval deklaráltuk.
2. Időzítő törlése: Minden egyes gépelési eseménynél (karakterleütésnél) első lépésként a clearTimeout(timeout) függvénnyel töröljük a már folyamatban lévő időzítőt.
3. Késleltetett indítás: Csak a törlés után indítunk egy új, 500 ms-os setTimeout időzítőt.

3.2 Egységtesztek

3.2.1 Tesztelési célkitűzések

A tesztelés elsődleges célja a backend üzleti logikájának és adatvalidációs folyamatainak izolált ellenőrzése. A tesztek garantálják, hogy a rendszerelemek (függvények) a specifikációnak megfelelően működnek, és a későbbi fejlesztések során nem keletkeznek regressziós hibák.

3.2.2 Alkalmazott technológiák

- Keretrendszer: Jest
- Módszertan: Automatikus egységtesztek izolált környezetben, függőségek mockolásával.
- Környezet: Node.js (fejlesztői függőségként telepítve).

3.2.3 Tesztelt komponensek leírása

A tesztek az alábbi modulokra terjednek ki:

Modul	Tesztelt funkciók	Ellenőrzés típusa
Adatvalidáció	Jelszókomplexitás, e-mail formátum, karakterhossz.	Szöveges bemenetek integritása és hibaüzenetek validálása.
Jogosultságkezelés	Admin szint ellenőrzése, hozzáférés megtagadása.	HTTP státuszkódok (403) és kivételkezelés.
Időkezelés	Nyitvatartási/időintervallum ellenőrzés.	HH:MM formátum és logikai sorrend (start < end).
Adatbázis logika	Felhasználó lekérése email/ID alapján, létezés ellenőrzése.	Mockolt aszinkron hívások és válaszobjektumok szerkezete.
Biztonság	Jelszó titkosítás (bcrypt) és összehasonlítás.	Hash-elés helyessége és hibás jelszó elutasítása.

3.2.4 Mockolási stratégia

Mivel az egységtesztek nem csatlakozhatnak valódi adatbázishoz, a `sql/querys.js` modul függvényeit **mockoltuk**. Ez lehetővé teszi a hálózati késleltetéstől mentes, gyors tesztfutást és a hibaágak (pl. nem létező felhasználó) könnyű szimulálását.

3.2.5 Tesztek futtatása

A tesztkörnyezet az „npm test” paranccsal indítható el.

A parancs lefuttatja a .test.js kiterjesztésű fájlokat, és részletes riportot ad a sikeres/sikertelen esetekről.

3.2.6 Tesztelési eredmények összefoglalása

A tesztek lefedik a kritikus üzleti logika 100%-át, beleértve:

- Pozitív esetek (helyes adatok elfogadása).
- Negatív esetek (helytelen formátumok, jogosultság hiánya).
- Szélsőséges esetek (üres tömbök, határértékek).

```
> beeready@1.0.0 test
> jest

 PASS  ./unit.test.js
  passwordTest
    √ érvényes jelszót elfogad (2 ms)
    √ hibát dob ha rövidebb mint 6 karakter (14 ms)
    √ hibát dob ha nincs nagybetű
    √ hibát dob ha nincs speciális karakter (1 ms)
    √ hibát dob ha nincs szám (1 ms)
    √ 400-as státuszkódot ad vissza hiba esetén
    √ több hibát is felsorol egyszerre (1 ms)
  emailTest
    √ érvényes email-t elfogad
    √ hibát dob @ nélkül
    √ hibát dob domain nélkül
    √ hibát dob TLD nélkül (2 ms)
    √ 400-as státuszkódot ad vissza (1 ms)
  admincheck
    √ nem dob hibát ha van elem a tömbben
    √ hibát dob ha üres a tömb (1 ms)
    √ 403-as státuszkódot ad vissza
  affectedRowscheck
    √ nem dob hibát ha érintett sorok száma > 0
    √ hibát dob ha affectedRows === 0 (1 ms)
  lengthtest
    √ elfogadja a határon belüli értéket (1 ms)
    √ hibát dob ha rövidebb mint min (1 ms)
    √ hibát dob ha hosszabb mint max
    √ 400-as státuszkódot ad vissza (1 ms)
  timetest
    √ elfogad érvényes időpontokat
    √ hibát dob ha start >= end
    √ hibát dob rossz formátum esetén (1 ms)
  getuserbyemail
    √ visszaadja a sort ha létezik a felhasználó (1 ms)
    √ hibát dob ha nem található az email
    √ 409-es státuszkódot ad vissza
  getuserbyid
    √ visszaadja a sort ha létezik a felhasználó
    √ hibát dob ha nem található az id (1 ms)
  checkuserexists
    √ nem dob hibát ha nem létezik a felhasználó
    √ hibát dob ha már létezik
    √ 409-es státuszkódot ad vissza
  encrypt és compare
    √ a hashelt jelszó különbözik az eredetitől (210 ms)
    √ helyes jelszóval nem dob hibát a compare (413 ms)
    √ rossz jelszóval hibát dob a compare (407 ms)
    √ a compare 403-as státuszkódot ad vissza (410 ms)
  isexistscheck
    √ mode=true: hibát dob ha már létezik (rows.length > 0) (2 ms)
    √ mode=true: nem dob hibát ha nem létezik (rows üres) (1 ms)
    √ mode=false: hibát dob ha nem található (rows üres)
    √ mode=false: nem dob hibát ha megtalálható
    √ 409-es státuszkódot ad vissza mindkét esetben (1 ms)
  share_code_generator
    √ visszaad egy stringet
    √ minden hívás más értéket ad
    √ csak base64url karaktereket tartalmaz

Test Suites: 1 passed, 1 total
Tests:       44 passed, 44 total
Snapshots:   0 total
Time:        1.805 s, estimated 3 s
Ran all test suites.
```

4. Unit teszt eredmények a konzolon

4 Továbbfejlesztési lehetőségek

4.1 Rövid távú fejlesztések (kb. 1 hónap)

4.1.1 Saját profilkép feltöltése

Jelenleg a felhasználók csak a 9 előre megadott avatar közül választhatnak. A saját kép feltöltése jelentősen személyesebbé teszi a felületet.

- Tárolás: lokális /uploads mappa, vagy felhőszolgáltatás (pl. Cloudinary – ingyenes szint elérhető)
- Validáció: csak kép típusú fájlok (jpg, png, webp), max. 2 MB méretkorlátozással

4.1.2 Sötét mód (Dark mode)

A felhasználók egyre inkább elvárják a sötét megjelenést, különösen esti tanulás esetén.

- CSS változók bevezetése a style.css-be (--bg-color, --text-color, --card-bg stb.)
- Egy toggle gomb a navbaron – állapot mentése localStorage-ba
- A Bootstrap dark utility osztályai kombinálhatók a meglévő stílusokkal

4.2 Közepes távú fejlesztések (1–3 hónap)

4.2.1 Haladáskövetés és statisztikák

A felhasználók motivációját erősen növeli, ha látják a fejlődésüket. A meglévő kvízeredmény-mentés (getquizresult) már alap, de vizuális megjelenítés nincs.

- Tanulókártya: hány kártyát nézett meg ma, ezen a héten, összesen
- Kvíz: korábbi eredmények listája, legjobb pontszám, átlag
- Grafikon: Chart.js vagy Recharts segítségével heti aktivitás megjelenítése
- Streak számlálás: hány egymást követő napon tanult a felhasználó

4.2.2 Flashcard – "Ment / Nem ment" szűrő

Kártyázás közben minden kártyánál a felhasználó két gomb közül választ: Ment vagy Nem ment. Ez az értékelés elmentődik az adatbázisban. A következő tanulási session indításakor a felhasználó kiválaszthatja, hogy az összes kártyát akarja-e átnézni, vagy csak azokat, amik nem mentek.

- Felhasználó megnyit egy paklit → megjelenik egy indítóképernyő két opcióval: Összes kártya (pl. 24 kártya) vagy Csak a nehezek (pl. 7 kártya, amik nem mentek)
- Kártyázás közben minden lap alján két gomb: Ment / Nem ment
- Session végén összesítő: "X kártyából Y-t tudtál – legközelebb csak a nehezeket mutatjuk"

4.2.3 Hive (közösségi) modul bővítése

A Hive modul már tartalmaz tartalom megosztást és keresést. Bővítési irányok:

- Értékelés (5 csillag) és komment funkció a megosztott kvízekhez és paklikhoz
- Kategóriák / témakörök szerinti szűrés (pl. matematika, biológia, angol)
- Legnépszerűbb ezen a héten szekció – a legtöbb kedvelés/letöltés alapján
- Felhasználói profil oldal – megosztott tartalmak listája, statisztikák

4.2.4 Barát jelölés és profil megtekintés

A felhasználók egymást barátként jelölhetik, és megtekinthetik egymás publikus profilját.

- Felhasználó keres egy másik felhasználónév alapján
- Barát kérést küld, a másik elfogadja vagy elutasítja
- Elfogadás után mindkettőjük barátlistájában megjelenik a másik
- A barát profiljára kattintva látható a publikus profilja

4.3 Hosszú távú fejlesztések (3+ hónap)

4.3.1 Gamifikáció – XP, szintek, jelvények

A motiváció fenntartása az egyik legnagyobb kihívás a tanulási alkalmazásoknál. A gamifikáció ezt hatékonyan segíti.

- XP (tapasztalati pont) rendszer: minden elvégzett kártya, kvíz, naptáreseményhez pont jár
- Szintek: XP alapján a felhasználó szintet lép (pl. Méhhallgató → Méhmester)
- Jelvények (badge): különleges célok elérésekor (pl. 7 napos streak, 100 kártya, első kvíz)
- Ranglista: iskolai / barátok között opcionálisan látható rangsor

4.3.2 Tanár–diák szerepkörök

Jelenleg az adminisztrátor és a normál felhasználó között nincs közbülső szerepkör. Oktatási környezetben hasznos lenne egy tanári fiók típus.

- Tanár létrehozhat osztályokat, és tanulókat adhat hozzájuk
- A tanár kvízt, feladatot rendelhet a tanulókhoz határidővel
- A tanár látja a tanulók eredményeit és haladását
- Automatikus értékelés: a kvíz beadásakor a rendszer azonnal pontozza és visszajelzést ad

4.3.3 Barátokkal chat

A barátként jelölt felhasználók egymással közvetlen üzenetet válthatnak az alkalmazáson belül – tanulási kérdések, pakli ajánlások, motiváló üzenetek küldésére.

4.3.4 Csoportos tanulás – Szobák

A felhasználó létrehozhat egy tanulószobát, ahová meghívja a barátait. A szobában mindenki ugyanazt a paklit vagy kvízt csinálja egyszerre, látják egymás haladását, és együtt tanulnak.

- A felhasználó létrehoz egy szobát → kap egy rövid belépési kódot (pl. BEE-4821)
- Elküldi a kódot a barátainak (chatben vagy kívülről)
- A barátok beírják a kódot → becsatlakoznak a szobába
- A szoba vezető kiválaszt egy paklit vagy kvízt → mindenki ugyanazt látja
- Tanulás közben egy oldalsávban látható, ki hol tart, ki hány kártyán van túl

5 Felhasználói dokumentáció

5.1 Lényegesebb funkciók

A BeeReady legfontosabb funkciói a következők:

- Tanulókártya paklik létrehozása, megosztása és közösségi keresése
- Kvízkészítő és kvízkitöltő modul változatos kérdéstípusokkal
- Teendőlista prioritáskezeléssel
- Naptár és órarend kezelése (A/B hetes rendszer)
- Hive közösségi modul tartalom böngészéséhez

5.1.1 Főoldal

A főoldal a bejelentkezés utáni alapértelmezett nézet, amely egy helyen biztosít hozzáférést a teendőlistához, az időzítőhöz, a naptárhoz és a statisztikához.

5.1.1.1 Teendőlista

5.1.1.1.1 Feladat hozzáadása

1. Írja be a feladat nevét a névmezőbe
2. Adjon meg leírást (nem kötelező)
3. Válasszon fontossági szintet: Alacsony / Közepes / Magas
4. Kattintson a hozzáadás gombra

5.1.1.1.2 Aktív feladatok kezelése

- Szerkesztés: a cím és a leírás szerkeszthetővé válik helyben, majd a Mentés gombbal elmenthető
- Kész: a feladatot befejezettnek jelöli, átkerül a kész feladatok közé
- A feladatkártyák háttérszíne jelzi a prioritási szintet

5.1.1.1.3 Kész feladatok kezelése\

- Vissza: visszahelyezi az aktív feladatok közé
- Törlés: véglegesen eltávolítja a feladatot

5.1.1.1.4 Statisztika widget

kördiagram, amely automatikusan frissül, és mutatja a feladatok megoszlását Magas / Közepes / Alacsony / Kész bontásban. Az egyes kategóriák száma szövegesen is megjelenik alatta.

5.1.1.1.5 A tanulási nézet használata

Adat	Korlát
Feladat neve	Legfeljebb 255 karakter
Fontossági szint	Magas, Közepes, Alacsony

5.1.1.2 Időzítő

Három üzemmód közül választható a legördülő listával.

5.1.1.2.1 Stopper mód

Az idő felfelé számlál a Start gombra kattintva.

5.1.1.2.2 Pomodoro mód

25 perces visszaszámláló, amely lejártakor értesítést küld.

5.1.1.2.3 Egyéni mód:

- Írja be a kívánt időt PP:MM formátumban (pl. 05:30)
- Kattintson a Start gombra

FIGYELEM: Csak számokat és kettőspontot fogad el a mező.

5.1.1.2.4 Gombok:

- **Start** – csak állva aktív
- **Stop** – csak futás közben aktív
- **Visszaállítás** – csak akkor aktív, ha az időzítő elindult vagy el van térítve az alapállapottól

5.1.1.2.5 Tanulással töltött idő

Az oldal alján folyamatosan mutatja az aktuális munkamenetben stopper és pomodoro módban töltött összes időt.

5.1.1.3 Naptár

havi nézet, a mai nap kiemelve jelenik meg. Az eseményes napokon egy kis pont látható, fölé húzva az egeret megjelenik az esemény neve.

5.1.1.3.1 Navigáció

A ‹ és › gombokkal lehet hónapot váltani.

5.1.1.3.2 Esemény megtekintése és hozzáadása:

- Kattintson bármelyik napra
- Megnyílik az esemény ablak, ahol láthatók az aznapi események
- Új esemény hozzáadásához: írja be az esemény nevét (kötelező), adjon meg leírást (nem kötelező), majd kattintson a Mentés gombra
- A dátumot az ablak tetején is módosíthatja, hogy más napra nézzen

5.1.1.3.3 Esemény törlése:

- Nyissa meg a kívánt napot
- Az esemény sorában kattintson a Törlés gombra

5.1.1.3.4 Technikai korlátok:

Adat	**Korlát**
Esemény neve	Legfeljebb 255 karakter, kötelező
Leírás	Nem kötelező

5.1.2 Tanulókártyák

A Tanulókártyák (flashcard) modul egy interaktív digitális kártyarendszer, amely segíti a felhasználókat az eredményes és hatékony tanulásban. A modul segítségével saját kártyacsomagokat (paklikat) hozhat létre, tölthet fel tartalommal, és különböző módokon gyakorolhatja az anyagot.

5.1.2.1 Paklik kezelése

A pakli (deck) a tanulókártyák gyűjteménye – hasonlóan egy tantárgyhoz vagy témához rendelt kártyacsomag. Az összes pakliját a főképernyőn láthatja egy listában.

5.1.2.1.1 Paklik létrehozása

1. Névmező megnyitása: A Jelenlegi paklijaid felirat felett látható szövegmezőbe kattintson.
2. Pakli elnevezése: Írja be az új pakli nevét (pl.: Történelem – 10. osztály). A név legfeljebb 200 karakter hosszú lehet.
3. Létrehozás: Kattintson a Pakli hozzáadása gombra, vagy nyomjon Entert.
4. Megjelenés: Az új pakli azonnal megjelenik a paklilistában.

A pakli neve után az alkalmazás automatikusan megjeleníti, hogy hány kártya található benne (pl.: 3 kártya).

5.1.2.1.2 Pakli sorrendjének megváltoztatása

A paklilistában fogd-és-ejtsd (drag-and-drop) módszerrel rendezheti át a paklikat:

- Fogja meg a pakli bal szélén látható :: jelzőt.
- Húzza a kívánt pozícióba.
- Engedje el – a rendszer automatikusan elmenti az új sorrendet.

5.1.2.1.3 Pakli szerkesztése

Egy pakli nevét, láthatóságát és megosztási kódját módosítani lehet:

1. Nyissa meg a paklit (kattintson rá).
2. A pakli nézetben kattintson a Szerkesztés gombra.
3. A megjelenő ablakban módosítsa a kívánt beállításokat:
    a. Pakli neve: az elnevezés tetszőlegesen módosítható.
    b. Publikus pakli: jelölőnégyzettel állítható be, hogy mások keressék-e a Hive-ban.
    c. Megosztási kód váltás: új megosztási kódot generálhat.
4. Kattintson a Mentés gombra a módosítások elmentéséhez.

5.1.2.1.4 Pakli törlése

Figyelem: a pakli törlése az összes benne lévő kártyát is véglegesen eltávolítja!

1. Nyissa meg a szerkeszteni kívánt paklit.
2. Kattintson a Szerkesztés gombra.
3. A megjelenő ablakban kattintson a Törlés gombra.
4. A pakli és az összes hozzá tartozó kártya törlődik, és visszakerül a paklilistára.

5.1.2.2 Kártyák kezelése

Miután megnyit egy paklit, megjelenik a kártyanézet, ahol az összes kártyát láthatja, szerkesztheti és kezelheti.

5.1.2.2.1 Pakli megnyitása

A paklilistában kattintson a kívánt pakli nevére. Megjelenik a kártyák listája és a paklihoz tartozó kezelőgombok.

5.1.2.2.2 Új kártya hozzáadása

1. Gomb megnyomása: Kattintson a Kártya hozzáadása gombra.
2. Elülső oldal: Az Elülső oldal mezőbe írja be a kérdést, fogalmat vagy szót (max. 255 karakter).
3. Hátsó oldal: A Hátsó oldal mezőbe írja be a választ, definíciót vagy magyarázatot (max. 400 karakter).
4. Mentés: Kattintson a Mentés gombra – a kártya azonnal megjelenik a listában.

5.1.2.2.3 Kártya szerkesztése

1. Keresse meg a szerkeszteni kívánt kártyát a listában.
2. Kattintson a kártya jobb oldalán lévő Szerkesztés gombra.
3. Módosítsa az elülső vagy a hátsó oldal tartalmát.
4. Kattintson a Mentés gombra.

5.1.2.2.4 Kártya törlése

1. Keresse meg a törölni kívánt kártyát a listában.
2. Kattintson a kártya jobb oldalán lévő Törlés gombra.
3. A kártya azonnal törlődik a pakliból.

5.1.2.2.5 Kártyák sorrendjének megváltoztatása

A kártyák sorrendje ugyanugy módosítható, mint a paklié: fogja meg a kártya bal oldalán látható :: jelzőt, majd húzza a kívánt pozícióba. Az új sorrend automatikusan mentésre kerül.

5.1.2.3 Tanulási mód

A tanulási mód az interaktív kártyavetítő funkció, ahol a pakli kártyáin egyenként lapozhat, és a kártyát megfordítva ellenőrizheti, hogy helyesen válaszolt-e.

5.1.2.3.1 A tanulás megkezdése

1. Nyissa meg a kívánt paklit.
2. Kattintson az Indítás gombra.
3. Válasszon indítási módot a felugró ablakban:
    a. Sorrendben: a kártyák a pakli eredeti sorrendjében jelennek meg.
    b. Véletlenszerűen: a kártyák véletlen sorrendben jelennek meg.
4. A tanulási nézet betöltődik, és megjelenik az első kártya elülső oldala.

5.1.2.3.2 A tanulási nézet használata

Gomb / Művelet	Leírás
Kártya megfordítása	Kattintson magára a kártyára. A kártya megfordul, és megjelenik a hátsó (válasz) oldal. Újra kattintva visszafordul.
Következő kártya	Kattintson a jobb oldali nyil gombra (>).
Előző kártya	Kattintson a bal oldali nyil gombra (<). Az első kártyánál ez a gomb inaktív.
Vissza a kártyákhoz	Kattintson a Vissza a kártyákhoz gombra a kártyalista nézethez való visszatéréshez.
Vissza a paklikhoz	Kattintson a Vissza a paklikhoz gombra a paklilistához való visszatéréshez.

5.1.2.4 Paklik megosztása

A BeeReady lehetővé teszi, hogy megossza paklijait másokkal egy egyedi megosztási kódon keresztül, illetve hogy más felhasználók paklijait másolja a saját gyűjteményébe.

5.1.2.4.1 Megosztási kód megismerése

Minden paklinak van egy egyedi, 8 karakteres megosztási kódja. A kód megtekintéséhez:

1. Nyissa meg a kívánt paklit.
2. Kattintson a Szerkesztés gombra.
3. A szerkesztő ablakban látható a Megosztási kód: #XXXXXXXX felirat.

5.1.2.4.2 Más pakli másolása megosztási kóddal

Ha valaki megosztott Önnel egy pakli kódot, az alábbi módon másolhatja a saját gyűjteményébe:

1. A paklilistán lévő szövegmezőbe írja be a kódot #XXXXXXXX formátumban (# jellel kezdve, 8 karakterrel).
2. Kattintson a Pakli hozzáadása gombra.
3. A rendszer automatikusan felismeri, hogy megosztási kódot adott meg, és másolatot készít a pakliból a saját fiókjában.

5.1.2.4.3 Megosztási kód cseréje

1. Ha biztonsági okokból meg szeretné változtatni a megosztási kódot:
2. Nyissa meg a szerkesztendő paklit.
3. Kattintson a Szerkesztés gombra.
4. Kattintson a Megosztási kód váltás gombra.
5. A régi kód azonnal érvénytelené válik, és egy új, véletlenszerű kód kerül hozzárendelésre.

5.1.2.4.4 Publikus pakli beállítása

Ha publikusra állít egy paklit, az megjelenik a Hive közösségi böngészőben, és bárki megtalálhatja, megtekintheti és lemásolhatja.

1. A Szerkesztés ablakban jelölje be a Publikus pakli jelölőnégyzetet.
2. Kattintson a Mentés gombra.
3. A pakli mostantól látható lesz a Hive-ban.

5.1.2.5 Technikai Korlátok

Adat / Funkció	Korlát / Megjegyzés
Pakli neve	Legfeljebb 200 karakter
Kártya elülső oldala	Legfeljebb 255 karakter
Kártya hátsó oldala	Legfeljebb 400 karakter
Megosztási kód formátuma	#XXXXXXXX (# + 8 karakter)
Idegen pakli szerkesztése	Nem lehetséges – csak megtekintés, indítás és másolás érhető el

5.1.3 Órarend

Az Órarend funkció segítségével áttekinthetően kezelheti és nyomon követheti a heti tanóráit és iskolai eseményeit.

- Nézet asztali gépen**:** Nagyobb képernyőn a rendszer egymás mellett, oszlopokba rendezve jeleníti meg a hét öt munkanapját (Hétfőtől Péntekig).
- Aktuális nap kiemelése**:** Az alkalmazás automatikusan felismeri az aktuális napot, és sárga kerettel (valamint a gomb kiemelésével) jelöli azt, hogy azonnal lássa a mai teendőit.

5.1.3.1 Hetek kezelése ("A" és "B" hét)

- Ha az intézményében váltakozó heti órarend van érvényben, lehetősége van "A" és "B" heteket kezelni.
- A képernyő bal felső sarkában található legördülő menüből választhatja ki, hogy az "A" hét vagy a "B" hét órarendjét szeretné megtekinteni.
- A hét átváltásakor a rendszer automatikusan betölti az ahhoz a héthez tartozó eseményeket, majd el is menti azt és a legközelebbi betöltéskor a legutóbb kiválasztott fog megjelenni.

5.1.3.2 Esemény (tanóra) hozzáadása

1. Új óra felvételéhez kövesse az alábbi lépéseket:
2. Kattintson a képernyő felső részén található sárga színű, **"+"** jelet ábrázoló gombra.
3. Ekkor felugrik az "Esemény hozzáadása" ablak.
4. Töltse ki az űrlapot a következő adatokkal:
    i. **Nap kiválasztása:** Válassza ki a legördülő menüből a napot (Hétfő - Péntek).
    ii. **Kezdés és Vége:** Adja meg az óra kezdetének és befejezésének pontos idejét.
    iii. **Tantárgy neve:** Írja be az óra nevét (pl. Matek).
    iv. **Helyszín:** Adja meg a termet vagy helyszínt (pl. B42).
5. Kattintson a zöld "Mentés" gombra az esemény rögzítéséhez, vagy a szürke "Mégse" gombra a megszakításhoz.

5.1.3.3 Meglévő esemény módosítása

Ha egy már felvett óra adatait (pl. elmarad, termet cserélnek) szeretné módosítani:

- Kattintson a módosítani kívánt esemény kártyájára az órarendben.
- A felugró ablakban megjelennek az esemény aktuális adatai, amelyeket átírhat.
- A módosítások véglegesítéséhez kattintson a **"Mentés"** gombra.

5.1.3.4 Esemény törlése

- Kattintson a törölni kívánt esemény kártyájára.
- A felugró ablak alján kattintson a piros **"Törlés"** gombra.
- Az esemény véglegesen eltűnik az órarendből.

5.1.3.5 Használat mobiltelefonon

A rendszer alkalmazkodik a kisebb képernyőkhöz is:

- Mobiltelefonon nem jelenik meg egyszerre az összes nap, hogy elkerülje a zsúfoltságot.
- Helyette a képernyő tetején megjelenik egy napválasztó sáv (H, K, SZ, CS, P gombokkal).
- Ezekre a betűkre kattintva válthat a napok között, és mindig csak a kiválasztott nap (aktív nap) óráit fogja látni.

5.1.3.6 Technikai korlátok

Adat / Funkció	Korlát	Megjegyzés
Tantárgy neve	Legfeljebb 100 karakter	
Terem neve	Legfeljebb 50 karakter	
Időpontok validációja	—	A tanóra végének időpontja nem lehet korábbi, mint a kezdete
Időtartam formátuma	—	HH:mm (24 órás formátum)
Napok kezelése	—	Hétfőtől péntekig (5 napos nézet támogatott)
Ütközéskezelés	—	A rendszer engedélyezi az azonos időpontra felvitt órákat, nem gátolja meg az ütközést

5.1.4 Kvíz

A Kvíz modul lehetővé teszi a felhasználók számára saját kérdéssorok létrehozását, szerkesztését, kitöltését és az eredmények nyomon követését. A felület reszponzív, így mobil és asztali nézetben is kényelmesen használható.

5.1.4.1 Kvízek kezelése

A Kvíz menüpontba belépve a saját és a mások által publikált (idegen) kvízeket kezelheti.

- Kvízek listázása: Az oldalon megjelennek a saját kvízek kártyaformátumban. Minden kártya tartalmazza a kvíz címét, a kérdések számát, a készítő felhasználónevét, és amennyiben van a leírást.
- Új kvíz létrehozása: A sárga "+" gombra kattintva nyitható meg a kvízkészítő felület.
- Idegen kvízek: Az "Idegen kvízek" kapcsolóval böngészhet más felhasználók publikus kvízei között, amelyeknél van saját kitöltés.
- Rendezés: A kvízkártyák a rajtuk található :: fogantyú segítségével szabadon átrendezhetők (drag-and-drop).

5.1.4.2 Kvíz létrehozása és szerkesztése

A kvíz létrehozásakor, szerkesztéskor megadható a cím, egy rövid leírás, valamint beállítható, hogy a kvíz publikus legyen-e, illetve a kérdések véletlenszerű sorrendben jelenjenek-e meg. Amennyiben publikusra állítjuk elérhetővé válik a Hive modulban, így ott kitölthető és menthető lesz.

A kérdések sorrendje módosítható a „::” elem segítségével.

FIGYELEM – Egy kvíz szerkesztésének mentésekor az összes előrehaladás(kitöltés és annak eredményei) elvesznek!

5.1.4.2.1 Kérdéstípusok

Négy különböző feladattípus közül választhat:

1. **Feleletválasztós kérdés:** Több válaszlehetőség adható meg, amelyek közül egy vagy több is megjelölhető helyesként. Minden válaszhoz egyedi pontszám rendelhető. Ideális igaz-hamis feladatok készítéséhez.
2. **Rövid válasz:** A felhasználónak szövegesen kell beírnia a pontos választ.
   Több helyes válasz is megadható, javasoljuk, hogy adjon meg több lehetőséget amennyiben nem számít, hogy kitöltéskor nagy vagy kis betűvel írja be a felhasználó a választ.
3. **Sorba rendező feladat:** A válaszokat a kitöltőnek a helyes sorrendbe kell húznia.
   Létrehozáskor állítsa be a helyes sorrendet a „::” elemek segítségével, a kvíz kitöltésekor a rendszer random sorrendben fogja megjeleníteni.
   Amennyiben egyéni pontszámot szeretne beállítani a kérdéshez pipálja be az egyéni pontozás űrlap elemet és adja meg a kívánt pontszámot.

FIGYELEM: egyéni pontszám állításnál a rendszer kiértékelésnél csak akkor adja meg a pontot ha az összes elem a helyére került, amennyiben nem a rendszer nem bontja a pontot hanem automatikusan nulla pont lesz az eredmény.

Amennyiben nem pipálja be az egyéni pontozás lehetőséget a rendszer automatikusan soronként értékel és egy sor egy pontot fog érni.

Példa: Ha hét sor van akkor az elérhető maximális pontszám 7 pont – ebből minden elrontott elem -1-et fog levonni.

4. **Kitöltő feladat (lyukas szöveg):** Egy szövegben a {válasz; pont} jelöléssel hozhatók létre kitöltendő mezők. Amennyiben nem megfelelő formátumban adja meg a válaszlehetőséget a rendszer automatikus 1 pontot fog beállítani a válasz értékének. Javasoljuk, hogy kerülje a szövegben a „{” és „}” karakterek használatát abban az esetben, ha ezzel nem a választ szeretné megadni. A válaszban lehet több „;” karakter is, azonban ebben az esetben is ajánlott elkerülni a válaszban a pontszám megadásán kívüli „;” használatát vagy figyelembe venni, hogy az utolsó „;” karakter a pontszám álljon.

5.1.4.3 Kvíz kitöltése

A kiválasztott kvíz az **"Indítás"** gombbal kezdhető el.

- A kitöltés során látható a kérdés szövege és a hozzá tartozó pontszám.
- Kilépés**:** A "Kilépés a kvízből" gombbal bármikor megszakítható a folyamat, de ilyenkor az előrehaladás elveszik.
- Beküldés**:** A válaszok megadása után a "Kvíz beküldése" gombbal fejezhető be a kitöltés.

5.1.4.4 Eredmények megtekintése

A kvízkártyákon található **"Eredmények"** gombra kattintva visszanézhetők a korábbi kitöltések.

- Megoldókulcs**:** Az eredményeknél bekapcsolható a "Megoldások megjelenítése", ami megmutatja a helyes válaszokat a felhasználó válaszai mellett.
- Kiértékelés**:** A rendszer jelzi a szerzett pontszámokat, és színekkel (zöld/piros) különbözteti meg a helyes és helytelen válaszokat.

5.1.4.5 Törlés

Lehetőség van a saját kvízek teljes törlésére, vagy idegen kvízek esetén csupán a saját kitöltési eredmények eltávolítására. A törlés előtt a rendszer minden esetben megerősítést kér.

5.1.4.6 Technikai korlátok

Adat / Funkció	Korlát	Megjegyzés
Kvíz címe	Legfeljebb 200 karakter	
Kérdés szövege	Legfeljebb 1000 karakter	
Válaszlehetőség szövege	Legfeljebb 1000 karakter	
Kérdések száma	Nincs szoftveres limit	Kvízenként nincs szoftveres limit, de a reszponzivitás miatt az optimális kezelhetőség javasolt
Pontszám értéke	Egész szám>0	Alapértelmezett értéke 1 pont
Kitöltő feladat formátuma	Speciális szintaxis szükséges	{válasz; pont}
Idegen kvíz szerkesztése	Nem lehetséges	Csak indítás és az eredmények törlése érhető el

5.1.5 Hive

A Hive a BeeReady közösségi tere, ahol más felhasználók nyilvános kvízeit és tanulókártya paklijait böngészheti, elindíthatja vagy elmentheti a saját gyűjteményébe.

5.1.5.1 Tartalom böngészése

Minden tartalom kártyaként jelenik meg. Egy kártyán látható:

- A tartalom típusa (KVÍZ vagy FLASHCARD jelölővel)
- A cím és a készítő neve
- Kvíznél: kérdések száma és leírás; flashcardnál: kártyák száma és megosztási kód
- Kedvelések száma (♥)

5.1.5.2 Szűrők és keresés

Szűrőgombok (fejlécben):

- **Összes** – minden nyilvános tartalom
- **Flashcardok** – csak paklik
- **Kvízek** – csak kvízek
- **Mentett** – a felhasználó által korábban elmentett elemek

A keresőmezőbe gépelve az alkalmazás automatikusan szűri a találatokat. Nem szükséges Entert nyomni.

5.1.5.3 Mentés és törlés

- Minden kártyán megjelenik egy **Mentés** gomb
- Mentés után a gomb felirata **Törlésre** vált – erre kattintva visszavonható a mentés
- Sikeres mentéskor és törlésnél az alkalmazás visszajelzést ad

5.1.5.4 Tartalom indítása

Az **Indítás** gomb csak más felhasználók tartalmain jelenik meg (a saját tartalmaidon nem).

- **Kvíznél:** az alkalmazás átnavigál a Kvíz modulba és azonnal elindítja a kitöltést
- **Flashcardnál:** az alkalmazás megnyitja a paklit a Tanulókártyák modulban

5.1.6 Profil kezelés

felhasználói fiók személyre szabását és az alapvető adatok karbantartását. A funkció az alábbiak szerint használható

5.1.6.1 Módosítható adatok

A profil oldalon a felhasználónak lehetősége van az alábbi információk frissítésére:

- Felhasználónév: A rendszerben megjelenő egyedi név módosítása.
- E-mail cím: A belépéshez és kapcsolattartáshoz használt cím frissítése.
- Jelszó: A fiók biztonsága érdekében új jelszó állítható be.
- Profilkép: A felhasználó az előre definiált avatarok közül választhat újat.

5.1.6.2 Profilkép (Avatar) választása

- Jelenleg kilenc előre definiált állatos avatar választását teszi lehetővé.
- A profilkép módosításához kattintson a profilkép-előnézetre, amely előhívja az avatar-választó modált.
- A felugró ablakban válassza ki a szimpatikus képet, amely ezután azonnal frissül a felületen.

5.1.6.3 Biztonság és ellenőrzés

- Jelszóvalidálás: Az új jelszó megadásakor a rendszer ellenőrzi a jelszó erősségét (hosszúság, kis- és nagybetűk, számok megléte).
- Adatok érvényessége: Minden módosítás előtt lefut egy hosszteljesítmény-ellenőrzés (min. és max. karakterhossz), megelőzve a hibás adatok mentését.

5.1.7 Admin felület

A felület célja a rendszerben regisztrált felhasználók és adminisztrátorok kezelése, adatainak módosítása és karbantartása.

5.1.7.1 A felület felépítése

A felület három fő részre tagolódik:

1. Navigációs sáv: Gyorselérés az adminisztrációs panelekhez és a kijelentkezéshez.
2. Adminisztrátorok kezelése: (Alapértelmezetten rejtett) Az adminisztrátori jogosultságú felhasználók listázása és hozzáadása.
3. Felhasználók kezelése: A rendszer összes regisztrált felhasználójának listája és szerkesztési lehetőségei.

5.1.7.2 Felhasználók keresése és listázása

A főoldalon található táblázat tartalmazza az összes felhasználót.

- Keresés: Használja a táblázat feletti keresőmezőt. A szűrés gépelés közben, 500ms késleltetéssel történik a felhasználónevek alapján.
- Adatok: Megtekinthető a felhasználó azonosítója (ID), felhasználóneve, e-mail címe, titkosított jelszava és profilképének elérési útja.

5.1.7.3 Felhasználói adatok módosítása és törlése

- Módosítás: A sor végén található "Módosítás" gombbal az adott felhasználó adatai szerkeszthetővé válnak.
- Törlés: A „Törlés” gombbal véglegesen eltávolítható a felhasználó a rendszerből.

5.1.7.4 Adminisztrátorok kezelése

Kattintson az „Adminok” gombra a felső menüsorban a panel megjelenítéséhez.

5.1.7.4.1 Új adminisztrátor hozzáadása:

1. Keresse meg az "Új adminisztrátor hozzáadása" szekciót.
2. Töltse ki a mezőket: Felhasználónév, E-mail, Jelszó.
3. Kattintson a zöld "Hozzáadás" gombra.

5.1.7.4.2 Adminok listája:

A felhasználókhoz hasonlóan itt is lehetőség van az adminisztrátorok adatainak keresésére, módosítására vagy jogosultságuk visszavonására (törlésére).

5.1.7.5 Biztonsági figyelmeztetések

- Jelszavak: A táblázatokban a jelszavak "hash" formátumban (titkosítva) jelennek meg a biztonság érdekében. Új jelszó sikeres mentése után a megadott jelszó hash formátumban fog megjelenni.
- Munkamenet vége: Ha befejezte a munkát, minden esetben használja a jobb felső sarokban található „Kijelentkezés” gombot az illetéktelen hozzáférés megakadályozása érdekében.

5.2 Hardver és szoftver szükségletek

Az alkalmazás futtatásához az alábbi feltételek szükségesek:

5.2.1 Szerver oldal

- Node.js 18+ futtatókörnyezet
- npm csomagkezelő (Node.js-szel együtt települ)
- Minimálisan ajánlott: 1 CPU mag, 512 MB RAM, 1 GB szabad lemezterület

5.2.2 Adatbázis

- MySQL vagy MariaDB (10.4+) relációs adatbázis-kezelő rendszer
- A szerver tesztelése MariaDB 10.4.32 verzión történt
- Szükséges adatbázis: BeeReady_db (utf8mb4 karakterkészlettel)
- Adatbázis kapcsolatkészlet: alapértelmezetten max. 10 egyidejű kapcsolat (konfigurálható)

5.2.3 Kliens oldal

Modern webböngésző JavaScript-támogatással (Chrome, Firefox, Edge, Safari)

Nincs szükség külön telepítésre a felhasználó eszközén

5.2.4 Hálózat

- A backend szerver alapértelmezetten a 4000-es porton fut (konfigurálható)
- A kliens és szerver közötti kommunikációhoz HTTP/HTTPS kapcsolat szükséges
- Rate limiting: alapértelmezetten 300 kérés/perc IP-címenként

5.2.5 Szoftverkörnyezet (fejlesztéshez)

- nodemon fejlesztői segédeszköz (automatikus újraindítás kódváltozáskor)
- phpMyAdmin vagy más MySQL kliens az adatbázis kezeléséhez (opcionális)

Főbb függőségek (automatikusan települnek npm install paranccsal):

- Express 5.x (webszerver keretrendszer)
- mysql2 (adatbázis-illesztőprogram)
- jsonwebtoken (JWT alapú hitelesítés)
- bcrypt (jelszó-titkosítás)
- cors, morgan, express-rate-limit

5.3 Telepítés és indítás

5.3.1 Rendszerkövetelmények

A szerver elindítása előtt az alábbi szoftvereknek telepítve kell lenniük:

Szoftver	Minimum	Megjegyzés
Node.js	v18.x LTS	Az alkalmazás futtatókörnyezete
npm	v9+	Node.js-szel együtt települ automatikusan
MySQL / MariaDB	v10.4+	Adatbázis-szerver (XAMPP is megfelelő)

5.3.2 Függőségek

5.3.2.1 Éles (production) függőségek

Az alábbi npm csomagok szükségesek az alkalmazás futtatásához. Az npm install parancs ezeket automatikusan telepíti.

Csomag	Verzió	Leírás
bcrypt	^6.0.0	Jelszavak biztonságos hash-eléséhez és ellenőrzéséhez (salt + hash algoritmus)
body-parser	^2.2.0	HTTP kérések JSON törzsének értelmezése és feldolgozása
bootstrap	^5.3.8	CSS keretrendszer; a backend statikusan kiszolgálja a /bootstrap útvonalon
cors	^2.8.5	Cross-Origin Resource Sharing (CORS) fejlécek kezelése
dotenv	^17.4.2	Környezeti változók betöltése a .env fájlból induláskor
express	^5.1.0	HTTP szerver- és útválasztó keretrendszer (5.x verzió)
express-rate-limit	^8.2.1	IP-alapú kéréskorlátok (globális + bejelentkezési végpont)
jsonwebtoken	^9.0.2	JWT tokenek generálása és hitelesítése (1 h / 7 napos lejárat)
morgan	^1.10.1	HTTP kérések naplózása konzolon és access.log fájlba
mysql	^2.18.1	Legacy MySQL adapter (egyes querys.js hívásokban)
mysql2	^3.15.3	Elsődleges MySQL2 driver – aszinkron, Promise-alapú kapcsolatkészlet
nodemon	^3.1.11	Automatikus szerver-újraindítás kódváltozáskor (dev módban is)

5.3.2.2 Fejlesztői (devDependencies) függőségek

Ezek a csomagok csak fejlesztés és tesztelés során szükségesek; éles szerverre nem kötelező telepíteni őket.

Csomag	Verzió	Leírás
jest	^30.3.0	JavaScript egységtesztelő keretrendszer (npm test)
nodemon	^3.1.10	Dev-módban automatikus újraindítás kódváltozáskor
@ljharb/tsconfig	^0.3.2	TypeScript alap-konfiguráció statikus típusellenőrzéshez

5.3.3 Telepítési útmutató

Az alkalmazás telepítése és indítása az alábbi lépésekkel történik:

1. A forráskód letöltése a repositoryból
2. A szükséges Node.js csomagok telepítése: npm install
3. Az adatbázis konfigurálása a .env fájlban

Változó	Alapértelmezett	Leírás
SERVER_IP	127.0.0.1	A szerver IP-je (helyi teszteléshez: 127.0.0.1)
PORT	4000	A HTTP szerver portja
DB_HOST	localhost	MySQL szerver elérhetősége
DB_USER	root	Adatbázis-felhasználó neve
DB_PASSWORD	(üres)	Adatbázis-jelszó
DB_NAME	BeeReady_db	Az adatbázis neve
DB_CONNECTION_LIMIT	10	Párhuzamos DB-kapcsolatok maximuma
JWT_SECRET	(kötelező!)	Token-aláírási titkos kulcs – generálás lentebb
RATE_LIMIT_WINDOW_MS	60000	Rate limit időablak ms-ban (alapból 1 perc)
RATE_LIMIT_MAX	300	Max kérés per IP az időablakon belül

JWT titkos kulcs generálása:

node -e "console.log(require('crypto').randomBytes(64).toString('hex'))"

4. Az adatbázis-sémák betöltése

   A backend/sql/beeready_db.sql fájl tartalmazza a teljes sémát és az alapadatokat. A szkript automatikusan létrehozza a BeeReady_db adatbázist.

   a) MySQL CLI-ből:

      mysql -u root -p < backend/sql/beeready_db.sql

   b) phpMyAdmin-ból (XAMPP):

      1. Nyisd meg: http://localhost/phpmyadmin
      2. Kattints az Importálás fülre
      3. Válaszd ki: backend/sql/beeready_db.sql
      4. Kattints a Végrehajtás gombra

5.3.4 Az alkalmazás indítása

5.3.4.1 Éles mód

npm start

5.3.4.2 Fejlesztői mód (nodemon – automatikus újraindítás):

npm run dev

Sikeres indítás után a konzol kimenete:

Szerver fut: http://127.0.0.1:4000/

5.4 Részletes bemutatás

Az alkalmazás első indításakor a felhasználónak regisztrálnia kell. A regisztráció során meg kell adni a felhasználónevet, az e-mail címet, egy biztonságos jelszót, és ki kell választani egy profilképet a kilenc elérhető avatar közül. Sikeres regisztrációt követően a felhasználó bejelentkezhet, és azonnal elérheti az összes funkciót.

6 Csapatmunka megvalósítása

A projekt két fejlesztő – Székely Balázs és Székely Hunor – közreműködésével valósult meg. A feladatok megosztása a csapattagok erősségei és érdeklődési területei alapján történt, biztosítva a hatékony együttműködést és az egyenletes munkaterhelést. A fejlesztés során mindkét csapattag mindkét területen (frontend és backend) dolgozott, azonban a fókuszterületek jellegükből adódóan eltértek egymástól.

6.1 Munkamegosztás

Székely Hunor elsősorban a frontend oldalon dolgozott. Az ő felelőssége volt a főoldal négy moduljának (teendőlista, időzítő, naptár, statisztika) megvalósítása, a navigációs sáv kialakítása, a Hive közösségi felület fejlesztése, valamint az admin felület elkészítése. Emellett Hunor végezte el a bejelentkezési és regisztrációs folyamat frontend részét, és közösen dolgoztak a profil kezelés megvalósításán.

Székely Balázs főleg a backend logika és az összetettebb funkciók fejlesztésére összpontosított. Az ő nevéhez fűződik a tanulókártyák teljes modulja (paklik, kártyák, tanulási mód), a kvízkészítő és kvízkitöltő rendszer (a négy kérdéstípussal és a kiértékelési logikával együtt), valamint az órarend modul kialakítása. Balázs felelős volt a REST API és a backend üzleti logika nagy részéért is, emellett szintén részt vett a bejelentkezés és a profil kezelés közös kialakításában.

6.2 Verziókezelés és együttműködés

A fejlesztés során Git verziókezelő rendszert alkalmaztunk. Ez lehetővé tette, hogy párhuzamosan, egymástól elkülönített ágakon dolgozhassunk, majd a kész részeket szabályozott módon integrálhassák a főágba. A verziókezelés segítségével minden változtatás nyomon követhetővé vált, és szükség esetén vissza lehetett állni egy korábbi, stabil állapotra. A merge konfliktusok feloldása és a kód áttekintése szintén a közös munka részét képezte, ami hozzájárult a kód egységességének megőrzéséhez.

A fejlesztési folyamat során rendszeresen egyeztettünk az aktuális feladatokról, a haladásról és a felmerülő technikai kihívásokról. A közös döntések – például az adatbázis-struktúra kialakítása és az API végpontok tervezése – megbeszélés útján születtek meg, biztosítva ezzel, hogy mindkét fél átlássa az egész rendszer működését. Ez a szemlélet különösen fontosnak bizonyult az olyan közösen használt komponensnél, mint a kettős validáció, a JWT munkamenet-kezelés vagy a központosított hibakezelés.

7 Összefoglalás

7.1 Leírás

A BeeReady projekt egy modern, digitális produktivitási és tanulástámogató alkalmazás, amely sikeresen ötvözi a feladatkezelés, az időmenedzsment és a tanulási eszközök funkcióit. Az alkalmazás letisztult, felhasználóbarát felületet kínál, amely mind technikai, mind esztétikailag egy korszerű webalkalmazást biztosít.

Összességében úgy gondoljuk, hogy a projekt során sikerült megvalósítani az első fejezetben kitűzött szakmai célokat. Az elkészült rendszer stabil alapokon nyugszik, jól strukturált, és alkalmas a további bővítésre is. A fejlesztés során külön figyelmet fordítottunk arra, hogy a felhasználói élmény és a funkcionalitás egyensúlyban legyen, így egy olyan alkalmazás született, amely nemcsak hasznos, hanem könnyen kezelhető is.

A munka során számos kihívással szembesültünk. Ezek közül kiemelkedett az egyes funkciók összehangolása, az adatkezelés megtervezése, valamint a felhasználói felület reszponzív kialakítása. Több esetben új megoldásokat kellett keresnünk, illetve utánajárnunk olyan technológiáknak, amelyekkel korábban nem dolgoztunk. Ezek a nehézségek azonban jelentősen hozzájárultak a fejlődésünkhöz.

A legtöbbet talán a problémamegoldás és a tervezés területén tanultuk. Megtapasztaltuk, hogy egy jól átgondolt struktúra mennyire fontos egy komplexebb alkalmazás esetében, valamint azt is, hogy a kisebb részfeladatokra bontás nagyban megkönnyíti a fejlesztési folyamatot. Emellett fejlődtek technikai készségeink is, különösen a webfejlesztés és az adatbázis-kezelés terén.

Végső soron a BeeReady projektet egy sikeres és hasznos munkának tartjuk, amely nemcsak egy működő alkalmazást eredményezett, hanem jelentős szakmai tapasztalatot is biztosított számunkra. A jövőben lehetőséget látunk a rendszer továbbfejlesztésére, új funkciók bevezetésére és a felhasználói visszajelzések alapján történő finomhangolásra is.

7.2 Fejlesztési eszközök

A fejlesztés során egy kliens–szerver architektúrán alapuló rendszer jött létre, amelyben a frontend HTML5, CSS3 és Vanilla JavaScript technológiákra, a backend Node.js és Express.js keretrendszerre, az adatréteg pedig MySQL adatbázisra épül. A hitelesítés JWT tokenek és Bcrypt titkosítás segítségével valósul meg, biztosítva a munkamenetek biztonságos kezelését.

7.3 Modulok és funkciók

Az alkalmazás hat fő modult tartalmaz: a főoldalt (teendőlista, időzítő, naptár, statisztika), a tanulókártyák rendszerét, a kvízkészítő és kitöltő modult, az órarendet, a Hive közösségi felületet, valamint a profil- és adminisztrációs felületet. A modulok egymástól jól elkülönítve, mégis egységes rendszerként működnek, köszönhetően a központosított response-kezelésnek, a kettős validációnak és a globális hibakezelő middleware-nek.

7.4 Minőségbiztosítás

A fejlesztés során felmerült hibákat – köztük a middleware sorrendből adódó hibakezelési problémát, a jogosulatlan adathozzáférési sebezhetőséget, az üres paklik kezelését és a Hive keresőmező optimalizálását – sikeresen azonosítottuk és javítottuk. A hibanapló tanúsága szerint a csapat körültekintően és felelősségteljesen kezelte a felmerülő technikai kihívásokat.

7.5 Továbbfejlesztési irányok

Az alkalmazás jól bővíthető architektúrával rendelkezik, amely lehetővé teszi a jövőbeli fejlesztések zökkenőmentes integrálását. A továbbfejlesztési tervek rövid, közepes és hosszú távú ütemezéssel biztosítják az alkalmazás folyamatos fejlődését – a saját profilkép feltöltésétől és a sötét módtól kezdve a gamifikációs elemeken és a tanár–diák szerepkörökön át egészen a csoportos tanulószobák megvalósításáig.

8 Ábrajegyzék

1. Projekt struktúra 10
2. Az adatbázis szerkezete 11
3. Általános API kérés folyamata 18
4. Unit teszt eredmények a konzolon 43

9 Irodalomjegyzék (Források)

Az alábbiakban az alkalmazás fejlesztése során felhasznált összes dokumentáció, könyvtár és eszköz forrása szerepel, modulok szerint csoportosítva. Az URL-ek az adott dokumentációs oldal közvetlen elérési útját jelölik.

9.1 Backend keretrendszer és szerver

[1] Node.js Foundation – *Node.js – hivatalos letöltési és dokumentációs oldal*. Elérhető: https://nodejs.org/en/docs [Utolsó megtekintés: 2025. 11. 14.]

[2] Express.js Contributors – *Express 5.x – API Reference*. Elérhető: https://expressjs.com/en/5x/api.html [Utolsó megtekintés: 2025. 12. 03.]

[3] Express.js Contributors – *Express.js – Using middleware*. Elérhető: https://expressjs.com/en/guide/using-middleware.html [Utolsó megtekintés: 2026. 01. 08.]

[4] Express.js Contributors – *Express.js – Error handling*. Elérhető: https://expressjs.com/en/guide/error-handling.html [Utolsó megtekintés: 2025. 12. 19.]

9.2 Adatbázis és kapcsolatkezelés

[5] sidorares (MySQL2 Contributors) – *MySQL2 – Quickstart és dokumentáció*. Elérhető: https://sidorares.github.io/node-mysql2/docs [Utolsó megtekintés: 2025. 10. 29.]

[6] sidorares (MySQL2 Contributors) – *MySQL2 – Connection Pools*. Elérhető: https://github.com/mysqljs/mysql/blob/master/Readme.md#pool-options [Utolsó megtekintés: 2025. 11. 07.]

[7] MariaDB Foundation – *MariaDB Server 10.4 – dokumentáció*. Elérhető: https://mariadb.com/kb/en/documentation/ [Utolsó megtekintés: 2025. 10. 31.]

9.3 Hitelesítés, JWT és jelszókezelés

[8] auth0 (jsonwebtoken Contributors) – *jsonwebtoken – npm csomag dokumentáció*. Elérhető: https://www.npmjs.com/package/jsonwebtoken [Utolsó megtekintés: 2025. 11. 22.]

[9] RFC 7519 – IETF – *JSON Web Token (JWT) – szabvány specifikáció*. Elérhető: https://datatracker.ietf.org/doc/html/rfc7519 [Utolsó megtekintés: 2025. 12. 11.]

[10] bcrypt Contributors – *bcrypt – npm csomag dokumentáció*. Elérhető: https://www.npmjs.com/package/bcrypt [Utolsó megtekintés: 2025. 11. 18.]

9.4 Rate limiting és HTTP naplózás

[11] express-rate-limit Contributors – *express-rate-limit – npm csomag dokumentáció*. Elérhető: https://www.npmjs.com/package/express-rate-limit [Utolsó megtekintés: 2026. 01. 15.]

[12] expressjs (morgan Contributors) – *morgan – HTTP request logger middleware for Node.js*. Elérhető: https://www.npmjs.com/package/morgan [Utolsó megtekintés: 2025. 12. 27.]

[13] motdotla (dotenv Contributors) – *dotenv – npm csomag dokumentáció*. Elérhető: https://www.npmjs.com/package/dotenv [Utolsó megtekintés: 2025. 11. 05.]

9.5 Frontend – CSS keretrendszer

[14] Bootstrap Contributors (Twitter/twbs) – *Bootstrap 5 – Get started (Bevezetés és telepítés)*. Elérhető: https://getbootstrap.com/docs/5.3/getting-started/introduction/ [Utolsó megtekintés: 2025. 10. 23.]

[15] Bootstrap Contributors (Twitter/twbs) – *Bootstrap 5 – Components (modálok, gombok, navigáció)*. Elérhető: https://getbootstrap.com/docs/5.3/components/modal/ [Utolsó megtekintés: 2025. 12. 09.]

[16] Bootstrap Contributors (Twitter/twbs) – *Bootstrap 5 – Utilities (segédosztályok)*. Elérhető: https://getbootstrap.com/docs/5.3/utilities/display/ [Utolsó megtekintés: 2026. 01. 21.]

9.6 Drag-and-drop rendezés – Sortable.js

[17] SortableJS Contributors – *SortableJS – Reorderable drag-and-drop lists (hivatalos dokumentáció)*. Elérhető: https://sortablejs.github.io/Sortable/ [Utolsó megtekintés: 2025. 11. 30.]

[18] SortableJS Contributors – *SortableJS – GitHub repository és API opciók*. Elérhető: https://github.com/SortableJS/Sortable#options [Utolsó megtekintés: 2026. 01. 16.]

[19] jsDelivr CDN – *SortableJS v1.15.0 – CDN elérési útvonal (felhasznált fájl)*. Elérhető: https://cdn.jsdelivr.net/npm/sortablejs@1.15.0/Sortable.min.js [Utolsó megtekintés: 2025. 11. 30.]

9.7 Tesztelés – Jest

[20] Meta Open Source (Jest Contributors) – *Jest – Delightful JavaScript Testing (főoldal)*. Elérhető: https://jestjs.io/ [Utolsó megtekintés: 2025. 12. 15.]

[21] Meta Open Source (Jest Contributors) – *Jest – Getting Started*. Elérhető: https://jestjs.io/docs/getting-started [Utolsó megtekintés: 2025. 12. 15.]

[22] Meta Open Source (Jest Contributors) – *Jest – Mock Functions (mockolási stratégia)*. Elérhető: https://jestjs.io/docs/mock-functions [Utolsó megtekintés: 2026. 01. 04.]

[23] Meta Open Source (Jest Contributors) – *Jest – Expect API (assertiönök)*. Elérhető: https://jestjs.io/docs/expect [Utolsó megtekintés: 2026. 01. 04.]

9.8 Fejlesztői segédeszközök

[24] remy (nodemon Contributors) – *nodemon – npm csomag dokumentáció*. Elérhető: https://www.npmjs.com/package/nodemon [Utolsó megtekintés: 2025. 10. 27.]

[25] cors Contributors – *cors – npm csomag dokumentáció (Cross-Origin Resource Sharing)*. Elérhető: https://www.npmjs.com/package/cors [Utolsó megtekintés: 2025. 11. 03.]

[26] expressjs (body-parser Contributors) – *body-parser – npm csomag dokumentáció*. Elérhető: https://www.npmjs.com/package/body-parser [Utolsó megtekintés: 2025. 11. 03.]

9.9 Webes szabványok és referenciák

[27] MDN Web Docs (Mozilla) – *localStorage – Web Storage API*. Elérhető: https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage [Utolsó megtekintés: 2025. 12. 22.]

[28] MDN Web Docs (Mozilla) – *sessionStorage – Web Storage API*. Elérhető: https://developer.mozilla.org/en-US/docs/Web/API/Window/sessionStorage [Utolsó megtekintés: 2025. 12. 22.]

[29] MDN Web Docs (Mozilla) – *fetch() – globális függvény (Fetch API)*. Elérhető: https://developer.mozilla.org/en-US/docs/Web/API/Window/fetch [Utolsó megtekintés: 2026. 01. 28.]

[30] MDN Web Docs (Mozilla) – *Date – JavaScript beépített objektum*. Elérhető: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date [Utolsó megtekintés: 2025. 12. 05.]

9.10 Egyéb felhasznált eszközök és inspiráció

[31] Coddy – *Ingyenes frontend és backend kurzusok*. Elérhető: https://coddy.tech/user/9hF7t8mRY1SaSkXTYhY6yEqvptU2 [Utolsó megtekintés: 2026. 04. 19.]

[32] Notion, Inc. – *Notion – Ötletek és tervezési minták*. Elérhető: https://www.notion.com [Utolsó megtekintés: 2025. 10. 08.]

[33] Pinterest, Inc. – *Pinterest – Weblap overlay vizuális inspiráció*. Elérhető: https://hu.pinterest.com/SzeHNR/weblap-overlay/ [Utolsó megtekintés: 2026. 02. 27.]

[34] OpenAI – *Sora AI – Képgenerálás mesterséges intelligenciával (avatárok készítéséhez)*. Elérhető: https://openai.com/sora/ [Utolsó megtekintés: 2025. 11. 22.]